

1.Data_Exploration

June 27, 2025

Myopia Study: Comprehensive Analysis, Modeling and Reporting

1 Table of Contents

- 1. Initialisation
 - Importing libraries and loading the myopia dataset for analysis
 - *Data Engineering*
 - * Summary
- 2. Predicting Model – Simplest models
 - functions
 - intro
 - 2.0.1. Logistic Regression
 - 2.0.2. Random Forest
 - 2.0.3. Gradient Boost
 - 2.1. Analysis
 - * 2.0.1. Model Performance (Before Data Enhancement)
 - Average Precision (AP) on the PR curve
 - Recall on Positive Class (`label=1`)
- 3. Data Exploration
 - functions
 - Clustering
 - * **Dimensionality Reduction for Interpretation**
 - * **Risk Profile Overlay**
 - * **Interpreting Distributions**
 - * **UMAP**
 - * **UMAP Projection: Visual Exploration of Risk Profiles**
 - * **From UMAP to Clustering: OPTICS & Reachability Plot**
 - * **Reachability Plot Analysis (OPTICS Clustering)**

- * **Optimizing Cluster Extraction via Epsilon Sweeping (OPTICS + DB-SCAN)**
 - * **OPTICS with UMAP: Clustering Interpretation**
 - 2.1. Statistic analysis
 - * 2.1.1. Univariate Analysis
 - * Univariate Analysis and Multivariate Visualization
 - * 2.1.2. Quant values
 - Conclusion and Synthesis
 - 4. Global Synthesis and Recommendations
 - Key Analytical Findings
 - UMAP + OPTICS Clustering Insights
 - Strategic Recommendations
-
- Next step : Predicting modeling (see 2.Predicting_modeling.ipynb)
-

2 1. Initialisation

2.0.1 Importing libraries and loading the myopia dataset for analysis

```
[1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import entropy, fisher_exact

import plotly.graph_objects as go
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import umap.umap_ as umap
import seaborn as sns
import shap
import plotly.express as px
from sklearn.metrics import (accuracy_score, roc_auc_score,
    ↳classification_report,
                                confusion_matrix, roc_curve, f1_score,
    ↳silhouette_score, precision_recall_curve, average_precision_score)

from sklearn.ensemble import RandomForestClassifier,
    ↳HistGradientBoostingClassifier
from sklearn.inspection import permutation_importance
```

```

from sklearn.cluster import OPTICS, cluster_optics_dbscan
import sys

import matplotlib.gridspec as gridspec
from itertools import cycle
from collections import Counter

from IPython.display import Markdown

from sklearn.calibration import CalibratedClassifierCV, calibration_curve

sys.path.append('src')

```

/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

```
from .autonotebook import tqdm as notebook_tqdm
```

```

[2]: df_train = pd.read_csv('myopia_train.csv', sep=',')
df_test = pd.read_csv('myopia_test.csv', sep=',')
display(df_train)
display(df_test)

```

	ID	STUDYYEAR	MYOPIC	AGE	GENDER	SPHEQ	AL	ACD	LT	VCD	\
0	498	1994	0	6	0	0.326	23.29	4.013	3.478	15.80	
1	439	1993	0	6	0	1.272	22.12	4.048	3.542	14.53	
2	359	1993	0	6	0	1.044	22.00	3.516	3.612	14.88	
3	267	1992	1	6	1	-0.207	21.82	3.904	3.450	14.47	
4	535	1990	0	6	0	-0.175	23.67	3.865	3.245	16.56	
..	...	...	...	...	...	...	...	...	...	...	
427	13	1993	0	6	0	2.185	22.19	3.490	3.570	15.13	
428	526	1993	0	7	0	1.848	22.74	3.636	3.336	15.77	
429	401	1991	0	7	0	0.691	23.25	3.702	3.668	15.88	
430	433	1995	0	6	1	0.983	22.31	3.464	3.526	15.32	
431	387	1995	0	6	1	0.483	21.91	3.288	3.598	15.03	

	SPORTHHR	READHR	COMPHR	STUDYHR	TVHR	DIOPTERHR	MOMMY	DADMY
0	12	0	5	1	3	16	1	1
1	6	5	4	0	6	29	1	1
2	12	7	2	0	10	35	1	0
3	20	4	0	3	4	25	1	1
4	9	0	5	0	14	24	1	0
..	...	...	...	...	...	...	...	...
427	8	2	3	0	3	15	0	0
428	8	2	3	2	8	26	0	0
429	15	2	2	1	10	23	0	1
430	4	1	1	0	3	8	0	0

```
431          5          0          1          0          7          9          0          0
```

```
[432 rows x 18 columns]
```

	ID	STUDYYEAR	MYOPIC	AGE	GENDER	SPHEQ	AL	ACD	LT	VCD	\
0	99	1993	0	6	1	0.290	23.50	3.786	3.584	16.13	
1	17	1995	0	6	1	1.033	23.33	3.730	3.322	16.28	
2	60	1991	0	6	1	0.596	22.45	3.488	3.710	15.25	
3	310	1993	0	7	0	0.661	22.15	3.824	3.654	14.68	
4	272	1994	0	6	1	1.127	22.19	3.716	3.280	15.20	
..	...	...	...	...	...	...	...	...	...	...	
181	572	1992	0	6	0	0.690	22.26	3.690	3.640	14.93	
182	72	1990	0	9	0	0.118	24.24	3.678	3.533	17.03	
183	395	1992	0	5	0	0.153	22.82	3.848	3.598	15.37	
184	91	1992	0	7	0	0.754	23.50	3.624	3.468	16.41	
185	148	1990	0	9	1	1.368	23.75	4.030	2.960	16.76	

	SPORTHR	READHR	COMPHR	STUDYHR	TVHR	DIOPTERHR	MOMMY	DADMY
0	7	4	2	0	14	30	1	0
1	4	5	1	1	3	23	0	0
2	5	2	0	1	10	19	0	1
3	21	4	2	5	3	34	0	1
4	10	0	2	0	5	9	0	0
..	...	...	...	...	...	...	...	...
181	14	10	8	1	10	59	1	1
182	7	0	0	4	4	16	1	1
183	15	2	2	1	7	20	1	1
184	26	0	0	0	8	8	0	0
185	9	2	1	5	7	30	0	0

```
[186 rows x 18 columns]
```

Columns : - **ID** : Incremental ID - **Study Year** : Year subject entered the study - **Myopic** : Myopia within the first five years of follow up - **Age** : Age at the first visit - **Gender** : Genre - **SPHEQ** : Spherical equivalent refraction - **AL** : Axial Length (mm) - **ACD** : Lens Thickness (mm) - **LT** : - **VCD** : - **SPORTHR** : Time spent engaging in sports/outdoor activities (hour/week) - **READHR** : Time spend for pleasure (hours/week) - **COMPHR** : Time spend playing video/computer games or working on the computer (hours/week) - **STUDYHR** : Time spend reading or study for school assignments (hours/week) - **TVHR** : Time spend watching television (hours/week) - **DIOPTERHR** : Composite of near-work activities (hours/week) - **MOMMY** : Was the subject's mother myopic ? - **DADMY** : Was the subject's father myopic ?

```
[3]: df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 432 entries, 0 to 431
Data columns (total 18 columns):
#   Column      Non-Null Count  Dtype
---

```

0	ID	432 non-null	int64
1	STUDYYEAR	432 non-null	int64
2	MYOPIC	432 non-null	int64
3	AGE	432 non-null	int64
4	GENDER	432 non-null	int64
5	SPHEQ	432 non-null	float64
6	AL	432 non-null	float64
7	ACD	432 non-null	float64
8	LT	432 non-null	float64
9	VCD	432 non-null	float64
10	SPORTHR	432 non-null	int64
11	READHR	432 non-null	int64
12	COMPHR	432 non-null	int64
13	STUDYHR	432 non-null	int64
14	TVHR	432 non-null	int64
15	DIOPTERHR	432 non-null	int64
16	MOMMY	432 non-null	int64
17	DADMY	432 non-null	int64

dtypes: float64(5), int64(13)

memory usage: 60.9 KB

```
[4]: print("train"), display(df_train.describe(include='all')), print("test"),
      display(df_test.describe(include='all'))
```

train

	ID	STUDYYEAR	MYOPIC	AGE	GENDER	\
count	432.000000	432.000000	432.000000	432.000000	432.000000	
mean	306.094907	1992.481481	0.131944	6.298611	0.488426	
std	177.969920	1.773648	0.338823	0.698821	0.500446	
min	1.000000	1990.000000	0.000000	5.000000	0.000000	
25%	150.500000	1991.000000	0.000000	6.000000	0.000000	
50%	308.000000	1993.000000	0.000000	6.000000	0.000000	
75%	456.250000	1994.000000	0.000000	6.000000	1.000000	
max	618.000000	1995.000000	1.000000	9.000000	1.000000	

	SPHEQ	AL	ACD	LT	VCD	SPORTHR	\
count	432.000000	432.000000	432.000000	432.000000	432.000000	432.000000	
mean	0.797419	22.481181	3.588583	3.545674	15.346921	11.902778	
std	0.631636	0.675306	0.227996	0.149772	0.664432	7.928321	
min	-0.699000	20.280000	2.772000	3.154000	13.410000	0.000000	
25%	0.441750	22.027500	3.436000	3.436000	14.900000	6.000000	
50%	0.741000	22.445000	3.596000	3.542000	15.330000	10.000000	
75%	1.044000	22.960000	3.744000	3.640000	15.832500	16.000000	
max	4.372000	24.560000	4.224000	4.112000	17.300000	45.000000	

	READHR	COMPHR	STUDYHR	TVHR	DIOPTERHR	MOMMY	\
count	432.000000	432.000000	432.000000	432.000000	432.000000	432.000000	
mean	2.662037	2.094907	1.437500	8.879630	25.368056	0.516204	

std	3.109247	3.022387	2.032401	5.766883	15.685458	0.500317
min	0.000000	0.000000	0.000000	0.000000	2.000000	0.000000
25%	0.000000	0.000000	0.000000	4.000000	14.000000	0.000000
50%	2.000000	1.000000	1.000000	8.000000	22.000000	1.000000
75%	4.000000	3.000000	2.000000	12.000000	33.000000	1.000000
max	20.000000	30.000000	15.000000	30.000000	91.000000	1.000000

DADMY

count	432.000000
mean	0.493056
std	0.500531
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000

test

	ID	STUDYYEAR	MYOPIC	AGE	GENDER \
count	186.000000	186.000000	186.000000	186.000000	186.000000
mean	317.408602	1992.075269	0.129032	6.301075	0.489247
std	180.108120	1.609005	0.336140	0.746667	0.501234
min	3.000000	1990.000000	0.000000	5.000000	0.000000
25%	168.500000	1991.000000	0.000000	6.000000	0.000000
50%	310.500000	1992.000000	0.000000	6.000000	0.000000
75%	485.750000	1993.000000	0.000000	6.000000	1.000000
max	616.000000	1995.000000	1.000000	9.000000	1.000000

	SPHEQ	AL	ACD	LT	VCD	SPORTHRR \
count	186.000000	186.000000	186.000000	186.000000	186.000000	186.000000
mean	0.809349	22.533011	3.555511	3.531651	15.446129	12.069892
std	0.614036	0.691717	0.234861	0.165023	0.660179	8.080705
min	-0.467000	19.900000	2.904000	2.960000	13.380000	0.000000
25%	0.468750	22.102500	3.410000	3.422000	15.000000	6.000000
50%	0.711500	22.530000	3.550000	3.536500	15.400000	10.000000
75%	1.008250	23.010000	3.690000	3.654000	15.840000	18.000000
max	4.228000	24.240000	4.250000	3.896000	17.030000	41.000000

	READHR	COMPHR	STUDYHR	TVHR	DIOPTERRHR	MOMMY \
count	186.000000	186.000000	186.000000	186.000000	186.000000	186.000000
mean	3.107527	2.129032	1.612903	9.107527	27.526882	0.483871
std	2.955389	3.142476	2.595348	5.618430	16.754566	0.501089
min	0.000000	0.000000	0.000000	0.000000	3.000000	0.000000
25%	1.000000	0.000000	0.000000	5.000000	16.000000	0.000000
50%	2.000000	1.000000	1.000000	8.000000	24.000000	0.000000
75%	4.750000	2.750000	2.000000	12.000000	36.000000	1.000000
max	14.000000	20.000000	15.000000	31.000000	101.000000	1.000000

	DADMY
count	186.000000
mean	0.510753
std	0.501234
min	0.000000
25%	0.000000
50%	1.000000
75%	1.000000
max	1.000000

[4]: (None, None, None, None)

```
[5]: print("Shape train:", df_train.shape)
      print(df_train.isnull().sum())
      print("%%%%%%%%%\nShape test:", df_test.shape)
      print(df_test.isnull().sum())
```

Shape train: (432, 18)

ID	0
STUDYYEAR	0
MYOPIC	0
AGE	0
GENDER	0
SPHEQ	0
AL	0
ACD	0
LT	0
VCD	0
SPORTHR	0
READHR	0
COMPHR	0
STUDYHR	0
TVHR	0
DIOPTERHR	0
MOMMY	0
DADMY	0

dtype: int64

%%%%%%%%%

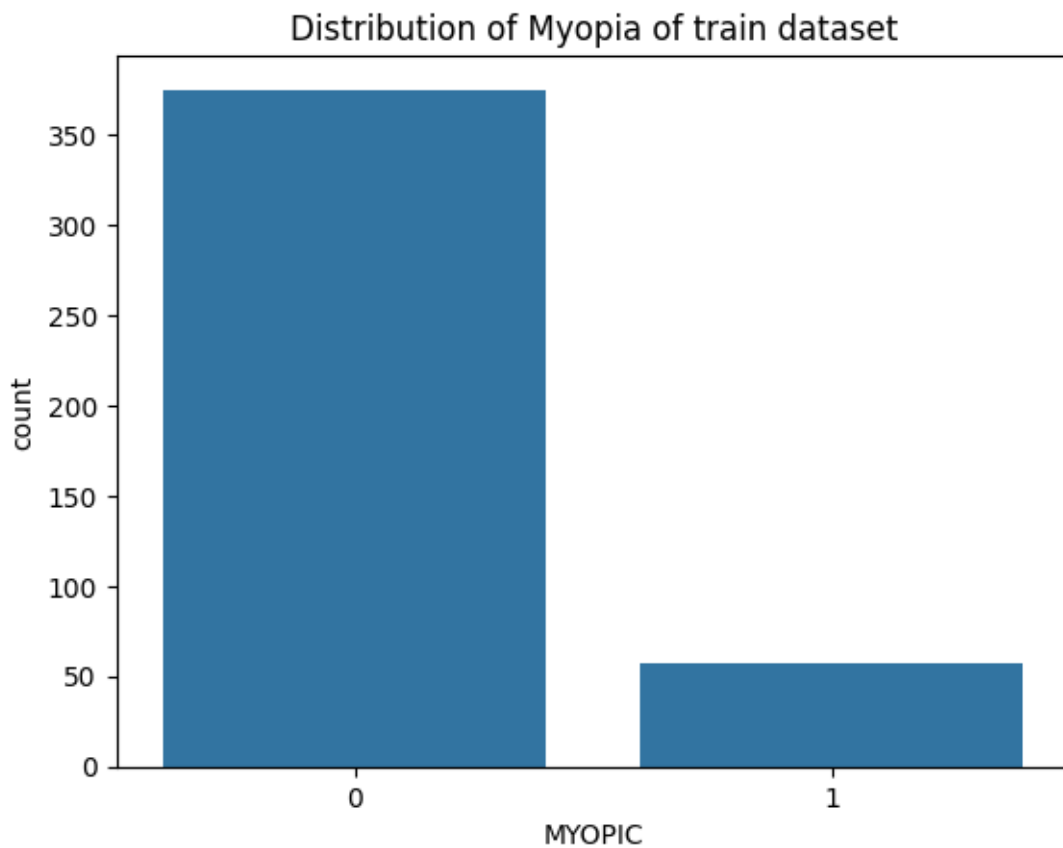
Shape test: (186, 18)

ID	0
STUDYYEAR	0
MYOPIC	0
AGE	0
GENDER	0
SPHEQ	0
AL	0
ACD	0
LT	0

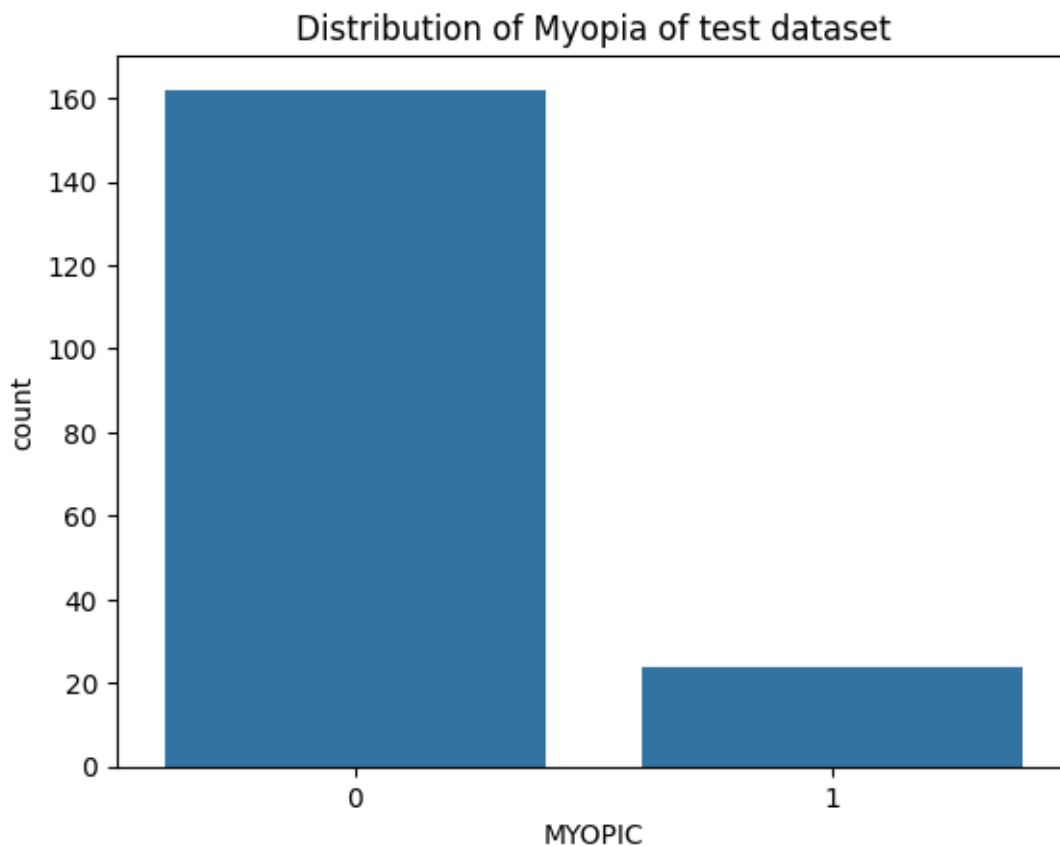
```
VCD          0
SPORTHRR     0
READHR       0
COMPHR       0
STUDYHR      0
TVHR         0
DIOPTERRR    0
MOMMY        0
DADMY        0
dtype: int64
```

```
[6]: sns.countplot(x='MYOPIC', data=df_train)
plt.title('Distribution of Myopia of train dataset')
plt.show()
print('Class distribution (%)')
print((df_train['MYOPIC'].value_counts(normalize=True) * 100).round(2))

sns.countplot(x='MYOPIC', data=df_test)
plt.title('Distribution of Myopia of test dataset')
plt.show()
print('Class distribution (%)')
print((df_test['MYOPIC'].value_counts(normalize=True) * 100).round(2))
```



```
Class distribution (%):
MYOPIC
0      86.81
1      13.19
Name: proportion, dtype: float64
```



```
Class distribution (%):
MYOPIC
0      87.1
1      12.9
Name: proportion, dtype: float64
```

Our dataset is **highly imbalanced**, with only around **11% positive cases** (class “yes”). In such a context, global metrics like **accuracy** or even **ROC-AUC** can be misleading, as they are dominated by the majority class.

The goal here is to **identify individuals at risk** and to **understand the contributing factors**, meaning we need to correctly predict the “yes” cases. This requires a strong focus on **recall for the positive class**, in order to **minimize false negatives**.

To properly assess model performance, we rely primarily on:

- **Average Precision (AP)**, a key metric for imbalanced settings as it balances precision and recall over the minority class,
- the **calibration curve**, to evaluate the reliability of predicted probabilities,
- the **Precision/Recall by threshold** curve, to analyze tradeoffs and help select decision thresholds based on clinical priorities (e.g., prioritizing recall vs. precision).

This evaluation framework provides a much more meaningful view of performance in a low-prevalence detection context like this one.

2.0.2 Data Engineering

```
[7]: df_train['PARENTSMY'] = ((df_train['DADMY']==1) | (df_train['MOMMY']==1)).
      ↪astype(int)
df_test['PARENTSMY'] = ((df_test['DADMY']==1) | (df_test['MOMMY']==1)).
      ↪astype(int)

df_train = df_train.drop(['MOMMY', 'DADMY', 'ID', 'STUDYYEAR'], axis=1)
df_test = df_test.drop(['MOMMY', 'DADMY', 'ID', 'STUDYYEAR'], axis=1)
```

```
[9]: df_train
```

```
[9]:
```

	MYOPIC	AGE	GENDER	SPHEQ	AL	ACD	LT	VCD	SPORTHR	\
0	0	6	0	0.326	23.29	4.013	3.478	15.80	12	
1	0	6	0	1.272	22.12	4.048	3.542	14.53	6	
2	0	6	0	1.044	22.00	3.516	3.612	14.88	12	
3	1	6	1	-0.207	21.82	3.904	3.450	14.47	20	
4	0	6	0	-0.175	23.67	3.865	3.245	16.56	9	
..	...	...	...	...	...	...	...	...	...	
427	0	6	0	2.185	22.19	3.490	3.570	15.13	8	
428	0	7	0	1.848	22.74	3.636	3.336	15.77	8	
429	0	7	0	0.691	23.25	3.702	3.668	15.88	15	
430	0	6	1	0.983	22.31	3.464	3.526	15.32	4	
431	0	6	1	0.483	21.91	3.288	3.598	15.03	5	

	PARENTSMY	SCREENHR	CLOSEHR
0	1	8	17
1	1	10	34
2	1	12	42
3	1	4	32
4	1	19	24
..	...	...	...
427	0	6	17
428	0	11	30
429	1	12	26
430	0	4	9

431 0 8 9

[432 rows x 12 columns]

Summary Key features were engineered to synthesize parental myopia risk and consolidate hours spent on screens or in close-up activities. Irrelevant or redundant variables were removed, resulting in a cleaner and more interpretable dataset. This step both streamlines later modeling and enhances overall scientific clarity.

3 2. Predicting Model - Simplest models

3.0.1 functions

```
[10]: def eval_model(model, X_train, y_train, X_test, y_test, seuil=0.5,
      ↪name='model', cv=5):
    model.fit(X_train, y_train)
    y_pred_proba = model.predict_proba(X_test)[:, 1]
    y_pred_label = (y_pred_proba > seuil).astype(int)
    print(f"\n===== {name} =====")
    print("Accuracy:", accuracy_score(y_test, y_pred_label))
    print("AUC:", roc_auc_score(y_test, y_pred_proba))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_label))
    print(classification_report(y_test, y_pred_label))

    ## Analysis
    test_results = X_test.copy()
    test_results['y_true'] = y_test
    test_results['y_pred'] = y_pred_label
    test_results['proba_pred'] = y_pred_proba
    return test_results, model

def eval_by_group(X, y_true, y_pred, group_col):
    groups = X[group_col].unique()
    for grp in groups:
        idx = X[group_col] == grp
        print(f"\n--- {group_col} = {grp} ---")
        print(classification_report(y_true[idx], y_pred[idx]))

def plot_precision_recall_threshold(model, X_train, y_train, X_test, y_test,
      ↪cv=5):
    # Entraînement
    model.fit(X_train, y_train)
    y_proba = model.predict_proba(X_test)[:, 1]

    # Courbes précision/rappel en fonction du seuil
    precisions, recalls, ths = precision_recall_curve(y_test, y_proba)
    plt.plot(ths, precisions[:-1], label='Precision')
```

```

plt.plot(thresholds, recalls[:-1], label='Recall')
plt.xlabel("Threshold")
plt.ylabel("Score")
plt.title("Precision / Recall by Threshold")
plt.legend()
plt.grid(True)
plt.show()

def plot_calibration_curve(model, X_train, y_train, X_test, y_test,
    method='isotonic', cv=5, label=None):
    # Calibration du modèle
    cal_model = CalibratedClassifierCV(model, method=method, cv=cv)
    cal_model.fit(X_train, y_train)

    # Prédiction calibrée
    y_cal_proba = cal_model.predict_proba(X_test)[:, 1]
    prob_true, prob_pred = calibration_curve(y_test, y_cal_proba, n_bins=10)

    # Affichage de la courbe de calibration
    plt.plot(prob_pred, prob_true, marker='o', label=label or f'{model.
    __class__.__name__} calibré')
    plt.plot([0, 1], [0, 1], 'k--', label='Perfect calibration')
    plt.xlabel("Predicted probability")
    plt.ylabel("Observed frequency")
    plt.title("Calibration curve")
    plt.legend()
    plt.grid(True)
    plt.show()

    return cal_model # Pour pouvoir utiliser le modèle calibré ensuite

def plot_precision_recall_curve(model, X_test, y_test, label=None):
    # Prédiction des probabilités
    y_proba = model.predict_proba(X_test)[:, 1]

    # Calcul des courbes
    precision, recall, _ = precision_recall_curve(y_test, y_proba)
    ap = average_precision_score(y_test, y_proba)

    # Affichage
    plt.plot(recall, precision, label=f'{label or model.__class__.__name__}
    (AP={ap:.2f})')
    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title("Precision-Recall Curve")
    plt.legend()

```

```
plt.grid(True)
plt.show()
```

3.0.2 intro

```
[11]: X_train = df_train.drop(['MYOPIC'], axis=1)
      X_test = df_test.drop(['MYOPIC'], axis=1)
      y_train = df_train['MYOPIC']
      y_test = df_test['MYOPIC']
```

3.0.3 2.0.1. Logistic Regression

```
[12]: # Logistic Regression
      print("="*30, 'Logistic Regression', "="*30)

      lr = LogisticRegression(solver='liblinear', max_iter=10000,
                              ↪class_weight='balanced', random_state = 20)
      test_results_lr, model_lr = eval_model(
          lr, X_train, y_train, X_test, y_test, name='Logistic Regression', seuil=0.5
      )

      explainer = shap.Explainer(model_lr, X_train)
      shap_values = explainer(X_test)

      shap.summary_plot(shap_values, X_test) # beeswarm

      plot_precision_recall_threshold(lr, X_train, y_train, X_test, y_test)
      cal_rf = plot_calibration_curve(lr, X_train, y_train, X_test, y_test,
                                     ↪label='Logistic Regression')
      plot_precision_recall_curve(lr, X_test, y_test, label="Logistic Regression")

      ===== Logistic Regression
      =====
```

===== Logistic Regression =====

Accuracy: 0.8333333333333334

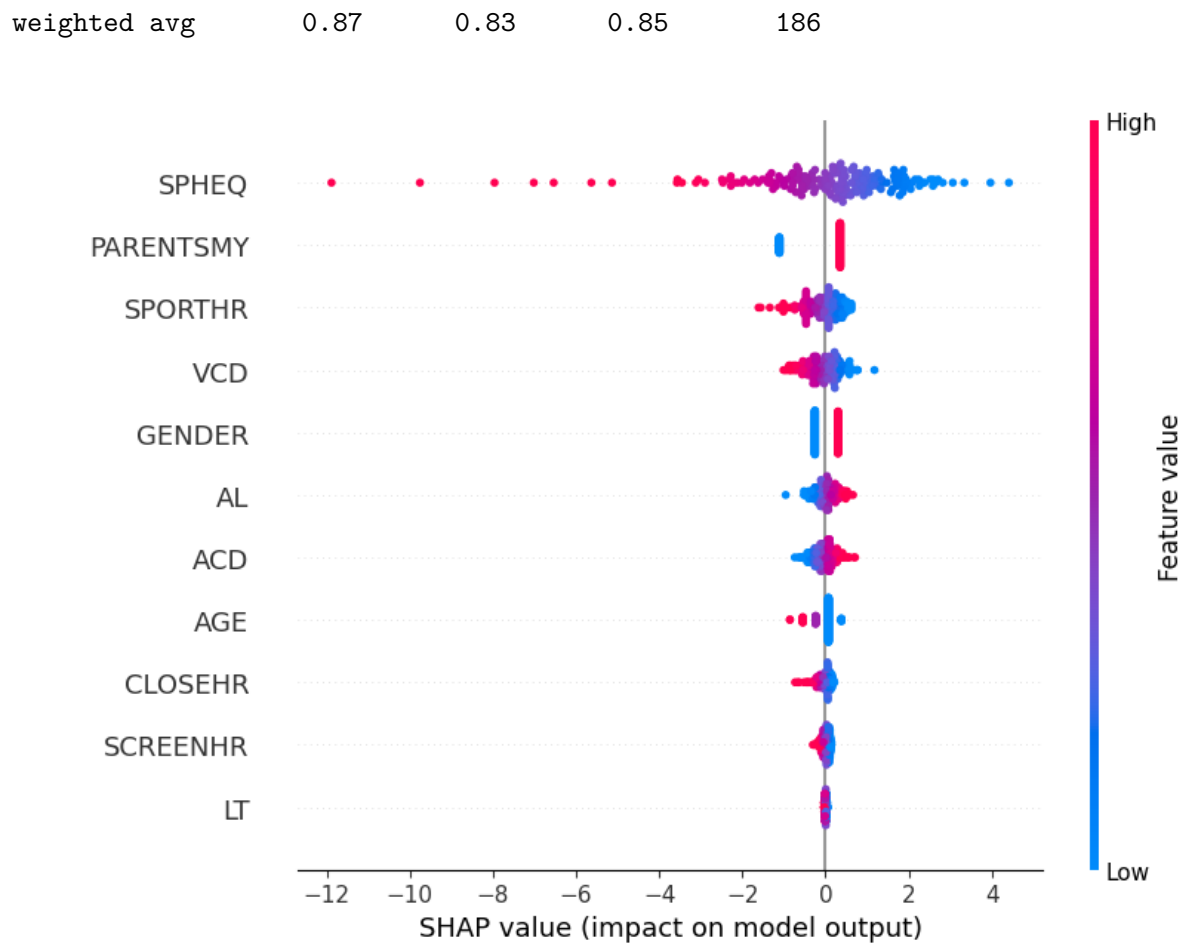
AUC: 0.8497942386831275

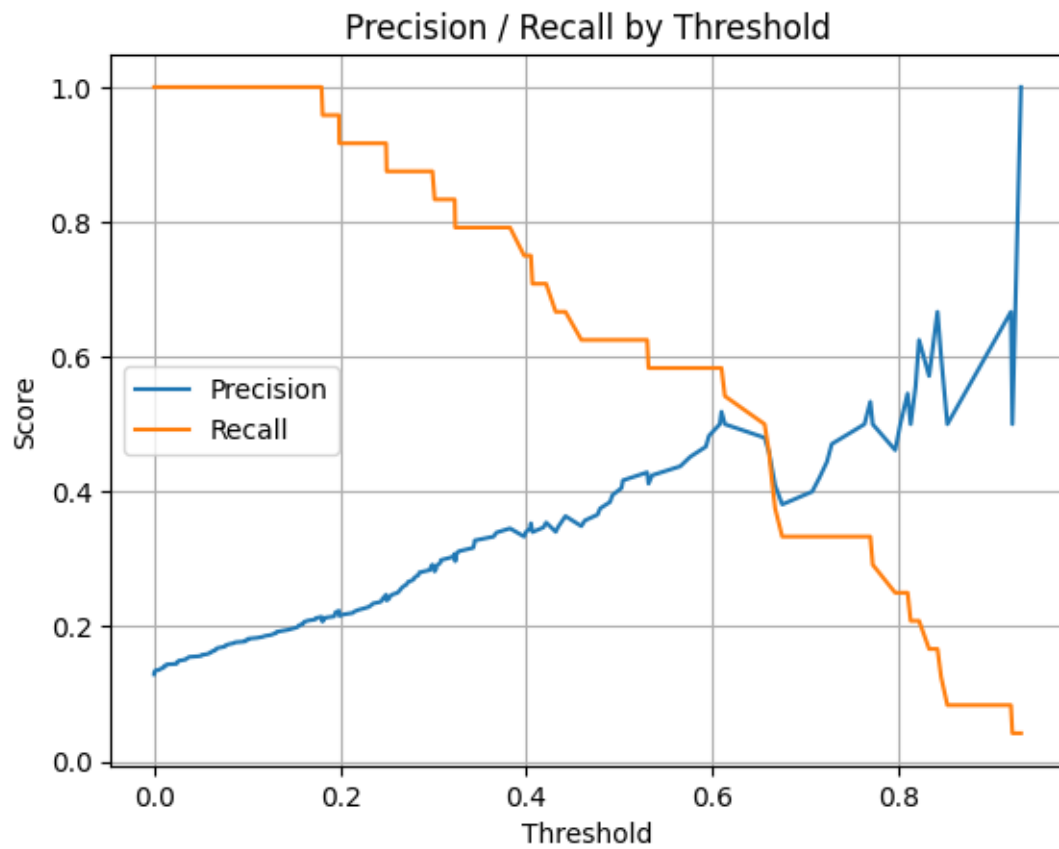
Confusion Matrix:

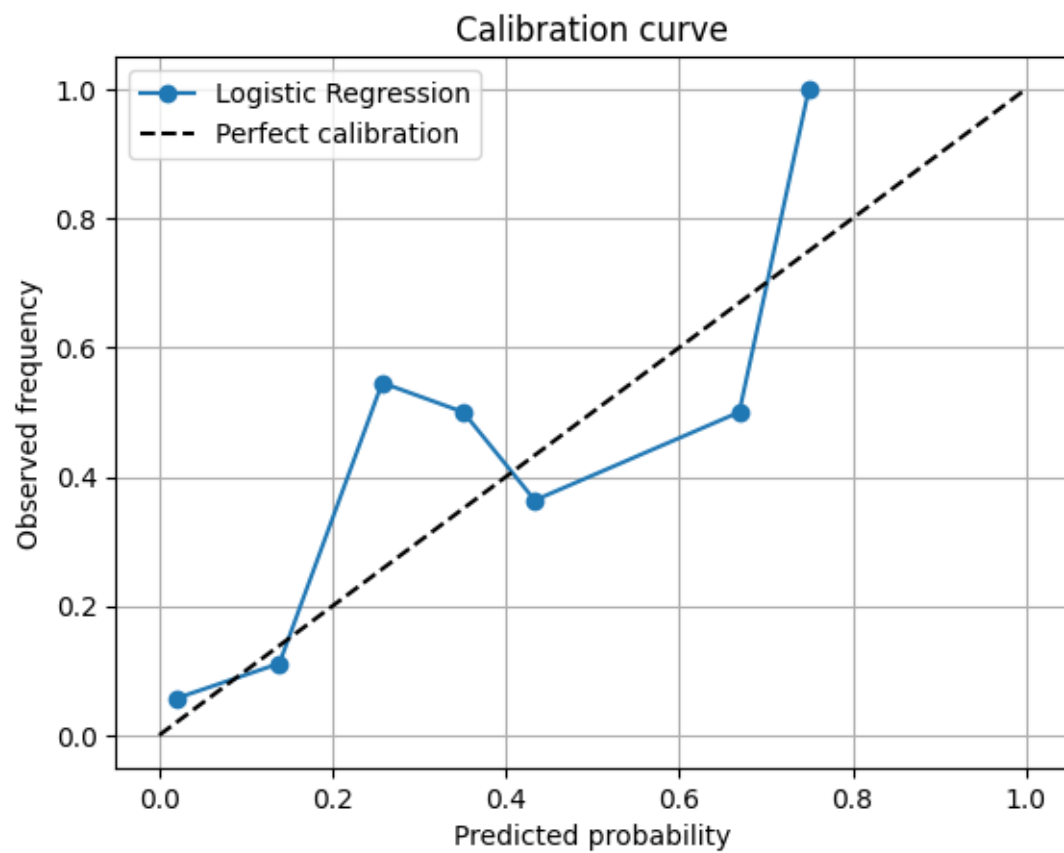
```
[[140  22]
```

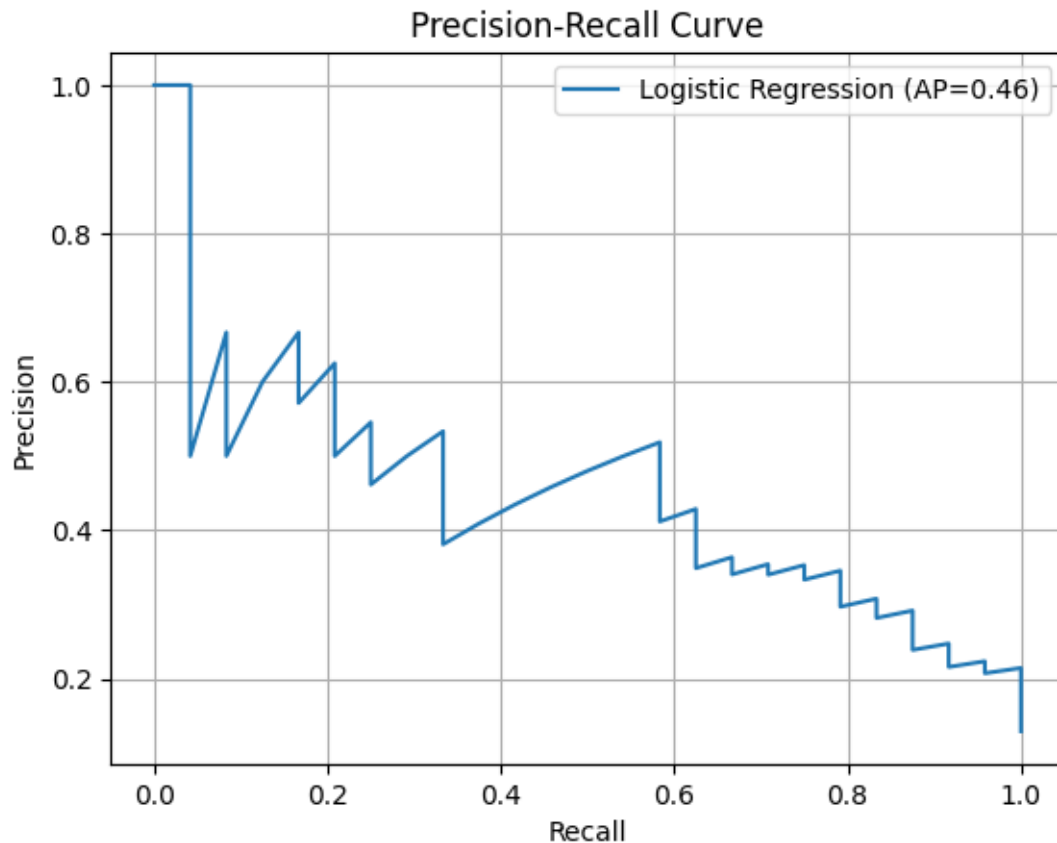
```
[ 9  15]]
```

	precision	recall	f1-score	support
0	0.94	0.86	0.90	162
1	0.41	0.62	0.49	24
accuracy			0.83	186
macro avg	0.67	0.74	0.70	186









3.0.4 2.0.2. Random Forest

```
[13]: # Random Forest
print("="*30, 'Random Forest', "="*30)
rf = RandomForestClassifier(n_estimators=100, class_weight='balanced',
    ↪max_depth=10, random_state=20)
test_results_rf, model_rf = eval_model(
    rf, X_train, y_train, X_test, y_test, name='Random Forest', seuil=0.5
)

# Feature importance
plt.figure(figsize=(8, 5))
feat_imp = pd.Series(model_rf.feature_importances_, index=X_train.columns)
feat_imp.sort_values(ascending=True).plot(kind='barh')
plt.title("Variable Importances (Random Forest)")
plt.xlabel("Importance")
plt.ylabel("Features")
plt.tight_layout()
plt.show()
```

```

plot_precision_recall_threshold(rf, X_train, y_train, X_test, y_test)
cal_rf = plot_calibration_curve(rf, X_train, y_train, X_test, y_test,
    ↪label='Random Forest')
plot_precision_recall_curve(rf, X_test, y_test, label="Random Forest")

```

===== Random Forest =====

===== Random Forest =====

Accuracy: 0.8709677419354839

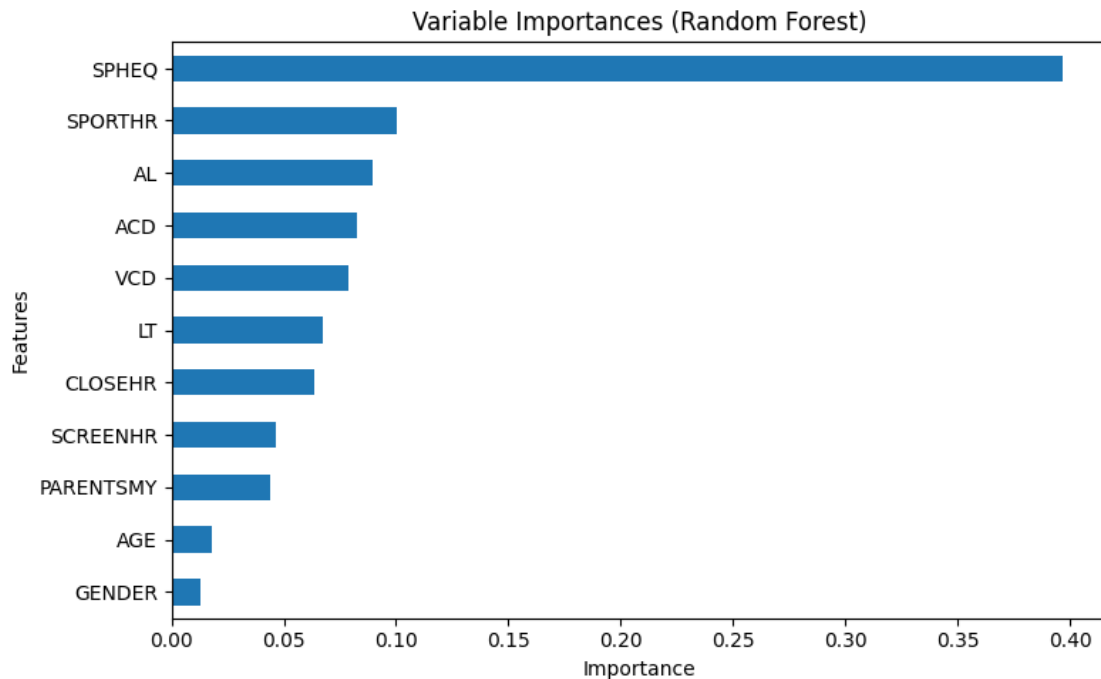
AUC: 0.8022119341563786

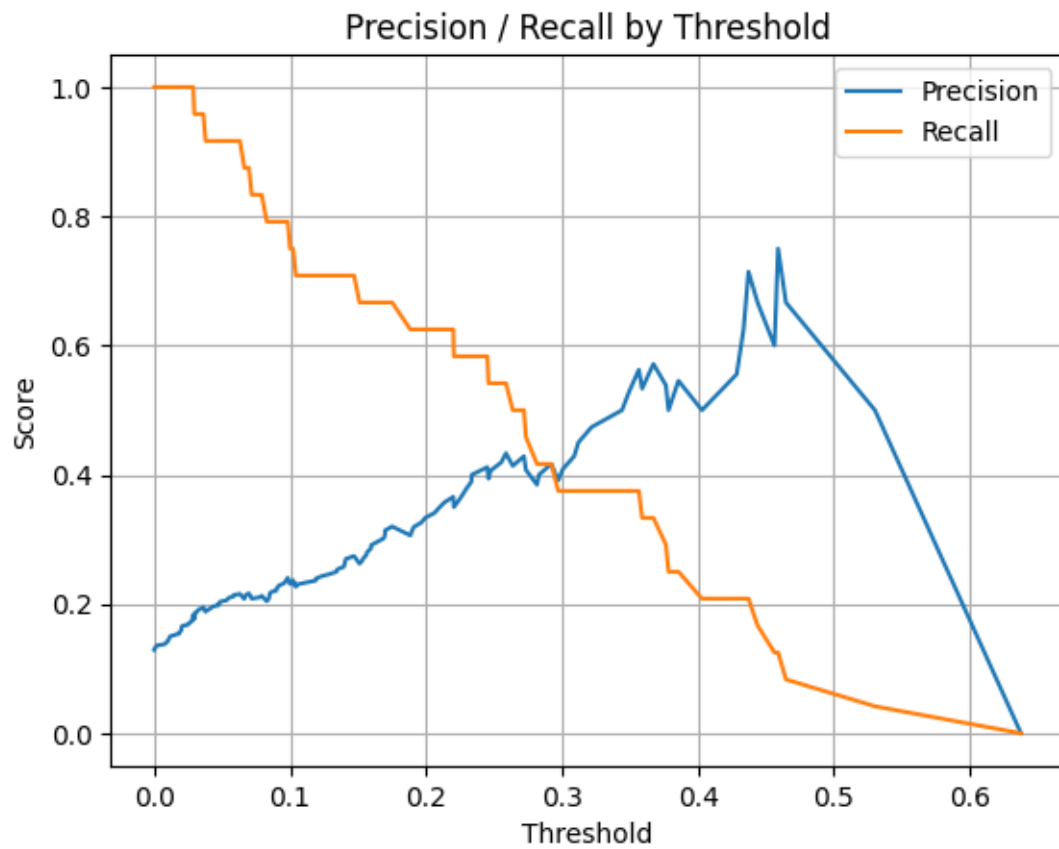
Confusion Matrix:

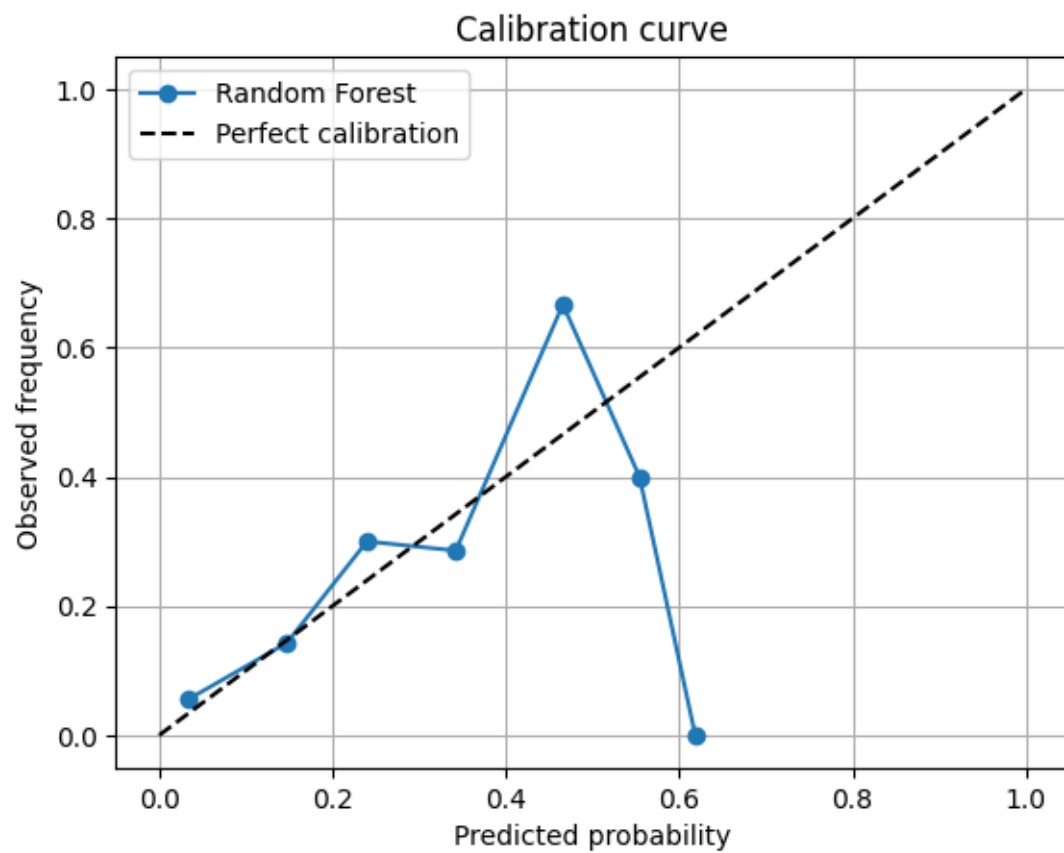
```
[[161  1]
```

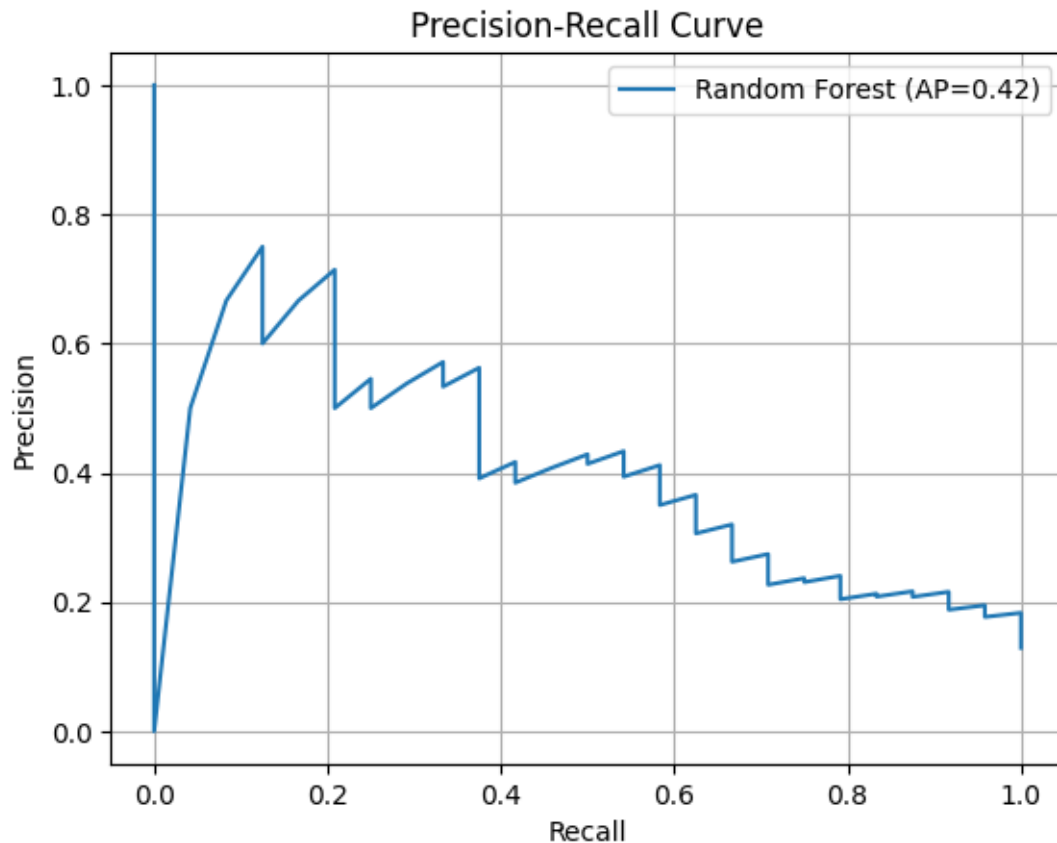
```
[ 23  1]]
```

	precision	recall	f1-score	support
0	0.88	0.99	0.93	162
1	0.50	0.04	0.08	24
accuracy			0.87	186
macro avg	0.69	0.52	0.50	186
weighted avg	0.83	0.87	0.82	186









3.0.5 2.0.3. Gradient boost

```
[14]: # GradientBoosting
print("="*30, 'GradientBoosting', "="*30)
hgb = HistGradientBoostingClassifier(class_weight='balanced', max_iter=100,
    random_state=20)
test_results_hgb, model_hgb = eval_model(
    hgb, X_train, y_train, X_test, y_test, name='GradientBoosting', seuil=0.5
)

# SHAP VALUES
explainer = shap.TreeExplainer(model_hgb)
shap_values = explainer.shap_values(X_test)

print("shap_values type:", type(shap_values))
if isinstance(shap_values, list):
    print("shape[0]:", np.array(shap_values[0]).shape)
    if len(shap_values) > 1:
        print("shape[1]:", np.array(shap_values[1]).shape)
else:
```

```

    print("shap_values:", np.array(shap_values).shape)
print("X_test:", X_test.shape)

if isinstance(shap_values, list) and len(shap_values) == 2 and np.
    ↪array(shap_values[1]).shape == X_test.shape:
    shap.summary_plot(shap_values[1], X_test)
else:
    # Certains cas (classification One-vs-Rest, régression, etc.)
    shap.summary_plot(shap_values, X_test)

plot_precision_recall_threshold(hgb, X_train, y_train, X_test, y_test)
cal_rf = plot_calibration_curve(hgb, X_train, y_train, X_test, y_test,
    ↪label='Gradient Boost')
plot_precision_recall_curve(hgb, X_test, y_test, label="Gradient Boost")

===== GradientBoosting =====

===== GradientBoosting =====
Accuracy: 0.8655913978494624
AUC: 0.7518004115226338
Confusion Matrix:
[[154   8]
 [ 17   7]]

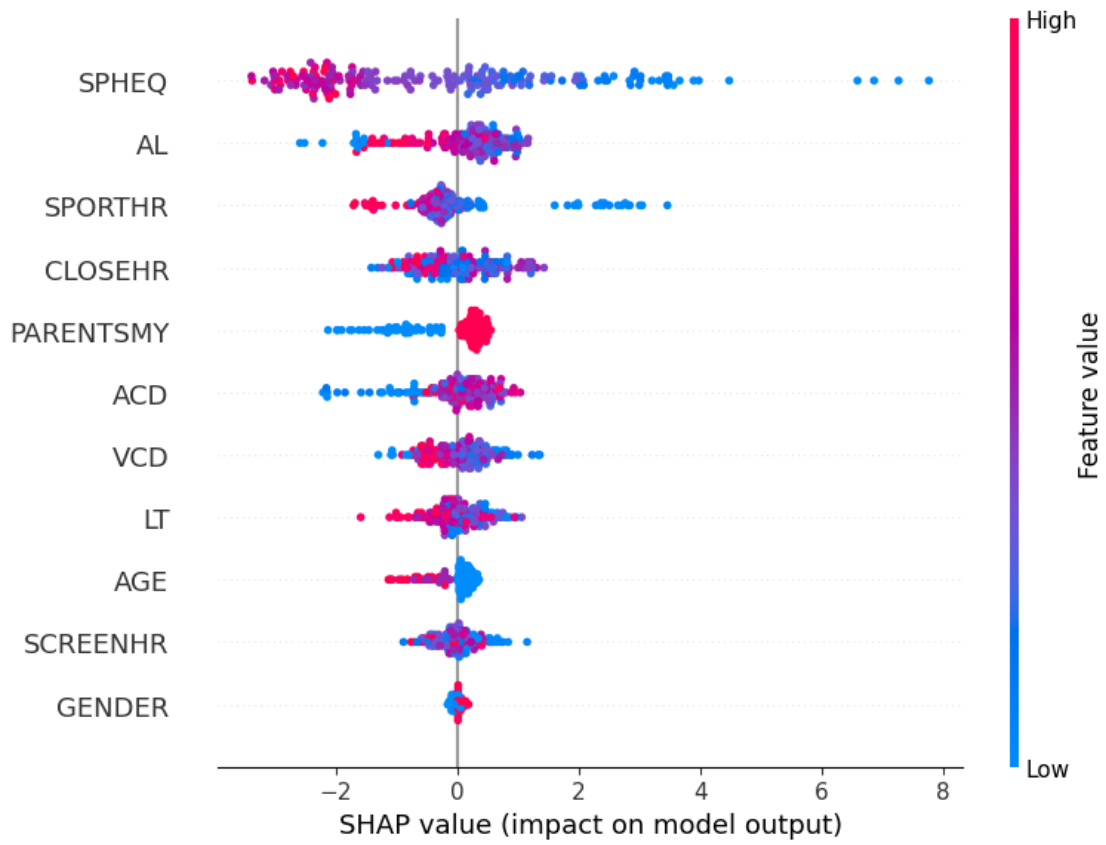
      precision    recall  f1-score   support

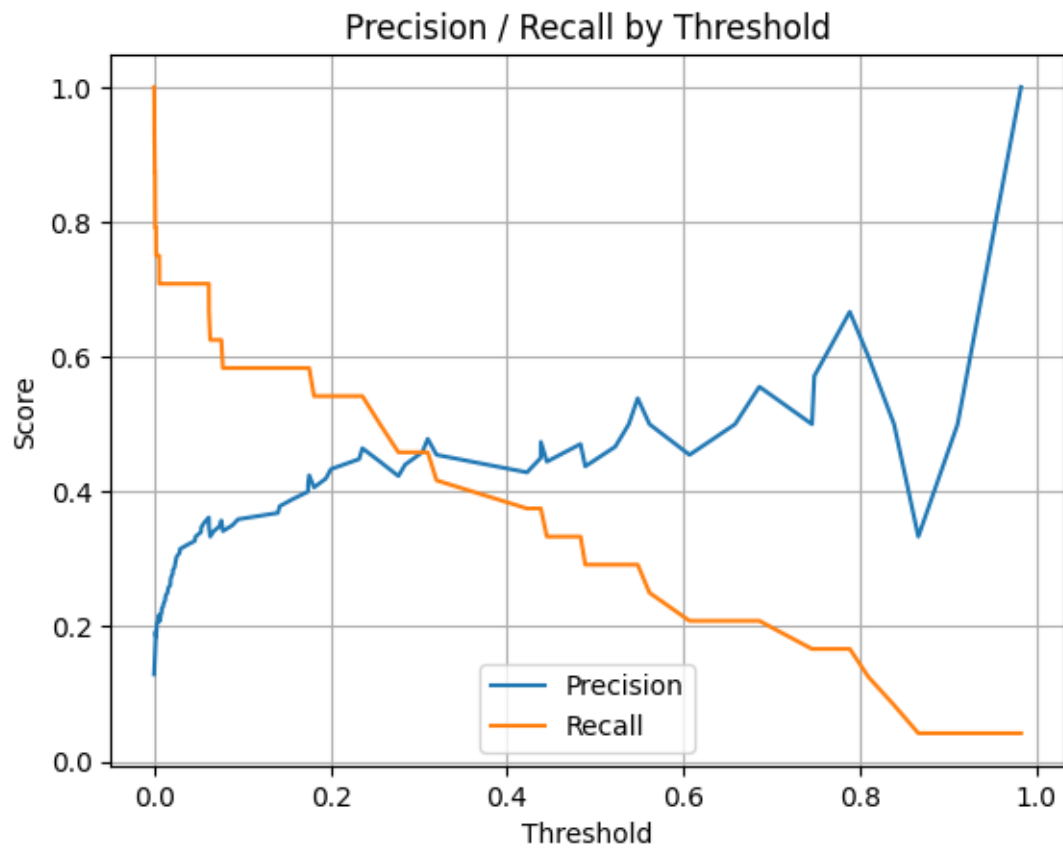
     0       0.90       0.95       0.92        162
     1       0.47       0.29       0.36         24

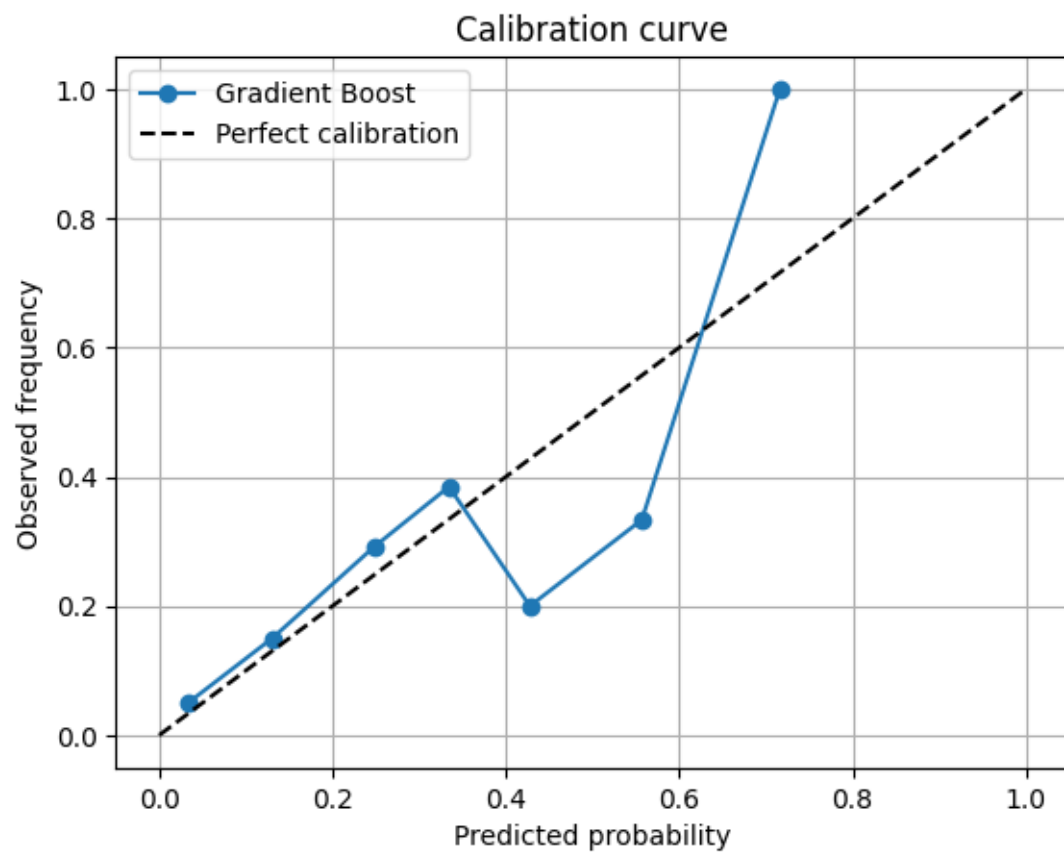
 accuracy                   0.87        186
 macro avg       0.68       0.62       0.64        186
weighted avg       0.84       0.87       0.85        186

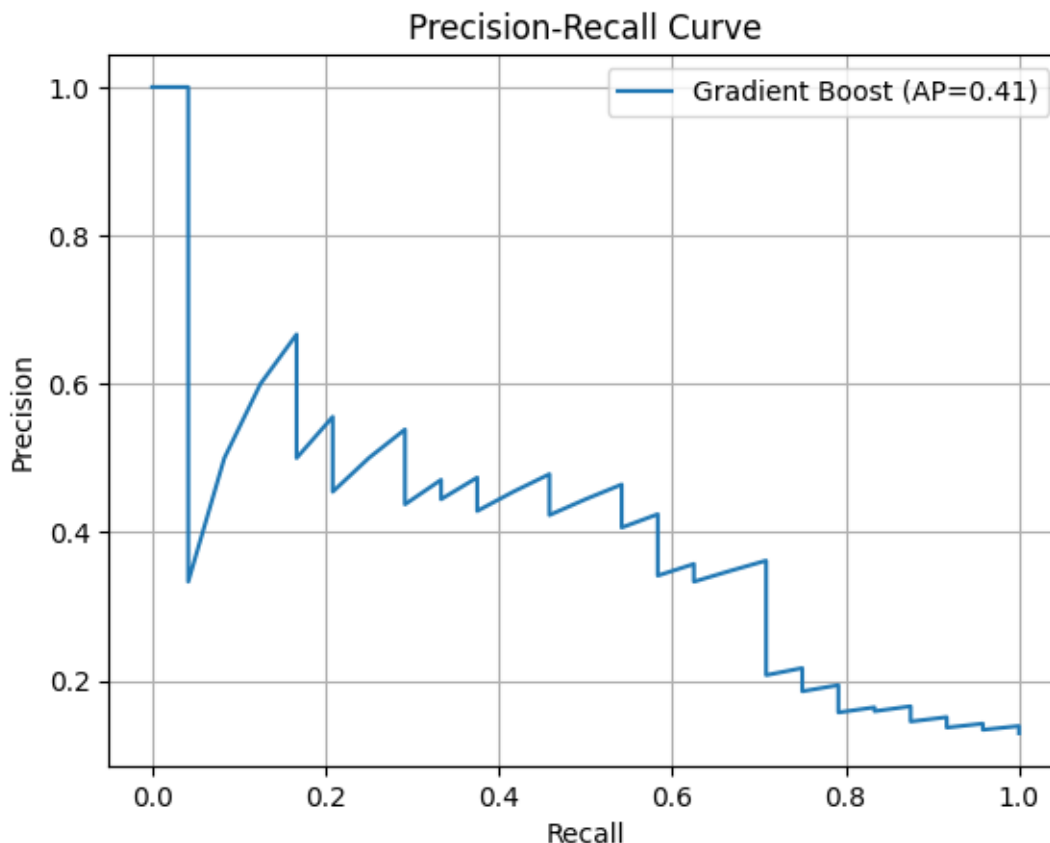
shap_values type: <class 'numpy.ndarray'>
shap_values: (186, 11)
X_test: (186, 11)

```









3.1 2.1. Analysis

3.1.1 2.0.1. Model Performance (Before Data Enhancement)

Because our dataset is highly imbalanced (only **11% positive cases**), we must carefully choose our evaluation metrics:

- The **F1-score** offers a compromise between precision and recall. However, in our context, we prioritize **high recall** to ensure we capture as many at-risk children as possible, while still keeping **precision** at a reasonable level to avoid too many false alarms.
- The **accuracy** metric is misleading with class imbalance. Similarly, the **ROC-AUC** tends to be overoptimistic and is less informative in this scenario.
- Therefore, we focus primarily on the **Precision-Recall (PR) curve**, **Average Precision (AP)**, **calibration curves**, and **precision-recall trade-offs across thresholds**.

With these metrics in mind, we evaluated three baseline models (before any data augmentation or optimization). Their performance was as follows:

Average Precision (AP) on the PR curve:

- Logistic Regression: 0.46
- Random Forest: 0.37
- Gradient Boosting: 0.37

Recall on Positive Class (label=1):

- Logistic Regression: 0.62
- Random Forest: 0.08
- Gradient Boosting: 0.21

These results show that even though all models are affected by the class imbalance, **Logistic Regression** performs noticeably better in identifying positive cases. However, all three models suffer from **low recall or poor calibration** when it comes to reliably detecting myopia risk.

Despite these limitations, their AP scores remain well above the prevalence baseline (11%), confirming that the models capture some useful predictive signal.

Across all three models, **SPHEQ (spherical equivalent)** consistently emerges as the most critical predictor of myopia risk, showing the highest SHAP values and feature importance scores. In Logistic Regression, SPHEQ dominates almost exclusively, whereas Random Forest and Gradient Boosting highlight a broader set of important variables. Specifically, **ocular biometric features** such as axial length (AL), anterior chamber depth (ACD), and vitreous chamber depth (VCD) rank highly in tree-based models, reflecting their strong clinical relevance. **Behavioral variables** like study hours (STUDYHR) and sport hours (SPORTHR) also show moderate impact, indicating lifestyle contributions to myopia risk. Additionally, **parental myopia (PARENTSMY)** consistently appears as an important predictor, aligning with known genetic influences. Overall, while SPHEQ remains the dominant feature across models, combining it with biometric and behavioral data enhances predictive performance and model robustness.

To improve upon this, we will now conduct a deeper **data exploration** phase to identify **important features**, uncover **key relationships**, and possibly reduce dimensionality. This will be followed by the application of techniques such as **SMOTE** and **parameter/hyperparameter optimization** to refine model performance — all of which will be addressed in the next section.

4 3. Data Exploration

The dataset is separated into numerical and categorical components to enable targeted exploratory analysis. This approach allows for tailored statistical summaries and visualizations, enhancing our understanding of both continuous variables and key risk subgroups before further modeling.

```
[ ]: df_train['SCREENHR'] = df_train['COMPHR'] + df_train['TVHR'] # Screens hour
df_train['CLOSEHR'] = df_train['READHR'] + df_train['STUDYHR'] +
    ↪df_train['DIOPTERHR'] # Activities with static eyes or close
df_train = df_train.drop(['COMPHR', 'TVHR', 'READHR', 'STUDYHR', 'DIOPTERHR'],
    ↪axis=1)

df_test['SCREENHR'] = df_test['COMPHR'] + df_test['TVHR'] # Screens hour
df_test['CLOSEHR'] = df_test['READHR'] + df_test['STUDYHR'] +
    ↪df_test['DIOPTERHR'] # Activities with static eyes or close
```

```
df_test = df_test.drop(['COMPHR', 'TVHR', 'READHR', 'STUDYHR', 'DIOPTERHR'],  
↪axis=1)
```

4.0.1 functions

```
[15]: def plot_optics_dbscan_clusters(X, labels, group, ax, title):  
    """  
    Affiche tous les points : couleur selon cluster, forme selon 'group'.  
    Les outliers sont en gris. Légende désactivée ici.  
    """  
  
    df = pd.DataFrame({  
        'x': X[:, 0],  
        'y': X[:, 1],  
        'label': labels,  
        'group': np.array(group)  
    })  
  
    marker_dict = {  
        'MYOPIC yes': 's',  
        'MYOPIC no': '^'  
    }  
  
    df_clustered = df[df['label'] != -1]  
    sns.scatterplot(  
        data=df_clustered,  
        x='x', y='y',  
        hue='label',  
        style='group',  
        markers=marker_dict,  
        palette='tab10',  
        ax=ax,  
        alpha=0.8,  
        edgecolor='black',  
        linewidth=0.3,  
        legend=False # Pas de légende ici  
    )  
  
    df_noise = df[df['label'] == -1]  
    if not df_noise.empty:  
        sns.scatterplot(  
            data=df_noise,  
            x='x', y='y',  
            hue='label',  
            style='group',  
            markers=marker_dict,
```

```

        palette={-1: 'lightgray'},
        ax=ax,
        alpha=0.5,
        legend=False
    )

    ax.set_title(title)
    ax.set_xlabel("t-SNE 1")
    ax.set_ylabel("t-SNE 2")

def get_myopic_score(summary_df):
    # Filtrer uniquement les clusters ayant au moins une personne
    non_empty_clusters = summary_df[summary_df['%_myopic_no'] > 0]
    non_empty_clusters = non_empty_clusters[non_empty_clusters['cluster'] != -1]
    if non_empty_clusters.empty:
        return 0 # Évite division par zéro
    return non_empty_clusters['%_myopic_no'].mean()

# Fonction de score combiné
def compute_scores(n_clusters_list, summary_eps, silhouettes, balances,
    ↪ valid_eps):
    # Préparation pour normalisation
    max_sil = max([silhouettes[i] for i in summary_eps])
    max_bal = max([balances[i] for i in summary_eps])
    max_myopic = max([get_myopic_score(df) for df in summary_eps.values()])

    results_ = []

    for indx, df_ in summary_eps.items():
        # Skip si bruit trop élevé
        #if df_['n_myopic_no'][df_['cluster'] == -1].iloc[0] >= 80:
        # continue

        sil = silhouettes[indx]
        bal = balances[indx]
        myopic = get_myopic_score(df_)

        norm_sil = sil / max_sil if max_sil else 0
        norm_bal = bal / max_bal if max_bal else 0
        norm_myo = myopic / max_myopic if max_myopic else 0

        final_score = (norm_sil + norm_bal + norm_myo) / 3

        results_.append({
            "index": indx,
            "eps": valid_eps[indx],

```

```

        "silhouette": sil,
        "balance": bal,
        "myopic_score": myopic,
        "score": final_score,
        "%_myopic_no" : valid_eps_scores[indx] ,
        "n_clusters": n_clusters_list[indx]
    })

    return pd.DataFrame(results_).sort_values(by="score", ascending=False)

```

4.0.2 Clustering

As a first exploratory step, we aim to identify potential clusters or patterns in the data based on various parameters. To facilitate this, we will apply **Principal Component Analysis (PCA)** to reduce the dimensionality while retaining as much information as possible, allowing us to visualise similarities and differences between **MYOPIC** classes.

```

[16]: X_train = df_train.drop('MYOPIC', axis=1)
      y = df_train['MYOPIC']

      group = df_train['PARENTSMY'].map({0: 'PARENTSMY yes', 1: 'PARENTSMY no'})

      scaler = StandardScaler()
      X_train_scaled = scaler.fit_transform(X_train)

      pca = PCA(n_components=2)
      X_train_pca = pca.fit_transform(X_train_scaled)

      display(pd.DataFrame(pca.components_, columns=X_train.columns, index=['PC1', 'PC2']))
      print ("Variance :", pca.explained_variance_ratio_[0] + pca.
            ↪ explained_variance_ratio_[1])

      plt.figure(figsize=(8,6))
      sns.scatterplot(x=X_train_pca[:,0], y=X_train_pca[:,1], hue=group, style=y,
            ↪ palette='Set1', alpha=0.8)
      plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)')
      plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)')
      plt.title("PCA : Visualisation regarding MYOPIA and parents")
      plt.legend(title='Parental Myopie / Status')
      plt.show()

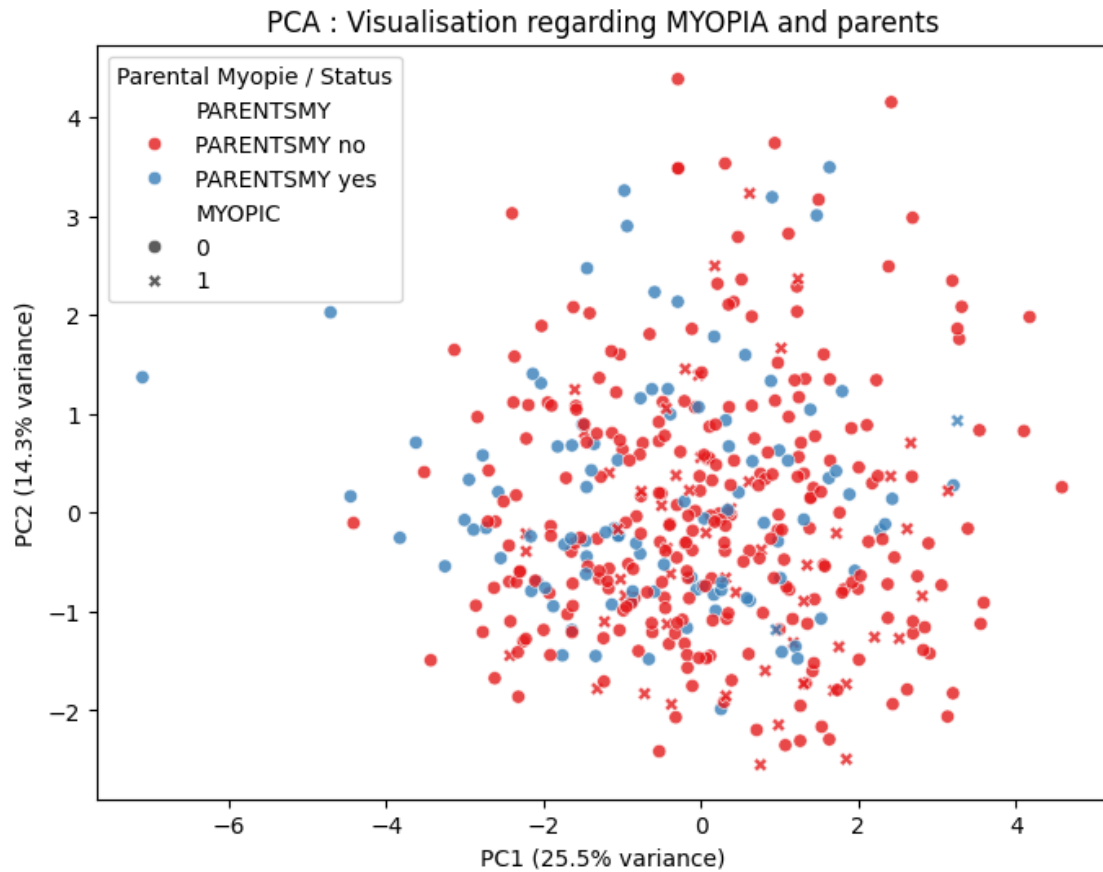
```

	AGE	GENDER	SPHEQ	AL	ACD	LT	VCD \
PC1	0.203758	-0.260833	-0.269532	0.537636	0.321875	-0.316087	0.507021
PC2	0.263270	-0.017573	0.063325	-0.112865	-0.193173	0.224350	-0.099040

	SPORTH	PARENTSMY	SCREENHR	CLOSEHR
PC1	0.128367	0.106474	0.121847	0.159694

PC2 0.305203 -0.057127 0.572897 0.625740

Variance : 0.3981257319910239



- **Dimensionality Reduction for Interpretation** The PCA projection reduces the high-dimensional feature space into two principal components: **PC1 (14.3%)** and **PC2 (25.5%)**, which together explain **only 40% of the total variance**. This relatively low cumulative variance indicates that the 2D projection **does not capture the full complexity of the dataset**.

Nevertheless, PCA remains valuable for visualizing the global data structure, allowing us to **identify potential trends, subgroups, or outliers** that may not be apparent in the original multi-dimensional space.

-
- **Risk Profile Overlay** Each point is annotated with two labels:
 - **Myopia status** (MYOPIC: 0/1)
 - **Parental myopia** (PARENTSMY: yes/no)

This dual-label visualization supports the exploration of the **clinical hypothesis** that **parental history**, a well-established risk factor for myopia, may help stratify children into **distinct risk**

categories.

- **Interpreting Distributions** Although there is **no clear linear separation** between myopic and non-myopic children in this 2D projection (suggesting the potential need for **non-linear modeling approaches**), the distribution reveals **localized groupings**.

In particular, children with **parental myopia** appear to cluster in certain regions, even in the absence of a strong global structure. To better capture these potential non-linear patterns, we will next apply **UMAP** for dimensionality reduction, followed by **OPTICS** and **DBSCAN clustering algorithms**. This approach could uncover **multifactorial risk interactions**, combining genetic predispositions and environmental influences. However, in the current PCA representation, we do not observe any **well-defined clusters or clear outliers**.

- **UMAP** Since PCA captured only **40% of the total variance**, and failed to clearly separate myopic and non-myopic children in a linear space, we extend our analysis with a **non-linear dimensionality reduction technique: UMAP (Uniform Manifold Approximation and Projection)**.

UMAP is particularly well-suited for high-dimensional biomedical data, as it preserves both **local neighborhoods** and **global structure**, enabling us to detect **complex, non-linear patterns** that linear methods may overlook.

By applying UMAP to the scaled training features, we generate a 2D projection that allows for **better visual exploration of risk profiles**, including the combined influence of **myopia status** and **parental history**. The resulting scatter plot highlights potential **latent clusters or risk regions**, offering clinical insights that may inform both **predictive modeling** and **hypothesis generation**.

```
[17]: data_train = df_train.copy()

reducer = umap.UMAP(n_components=2, random_state=42)

X_train_umap = reducer.fit_transform(X_train_scaled)

plt.figure(figsize=(8,6))
sns.scatterplot(x=X_train_umap[:,0], y=X_train_umap[:,1], hue=y, style=group,
               palette='coolwarm', alpha=0.7)
plt.title('UMAP : identification of risk profiles')
plt.xlabel('UMAP axis 1')
plt.ylabel('UMAP axis 2')
plt.legend(title='Myopia / Parents')
plt.show()
```

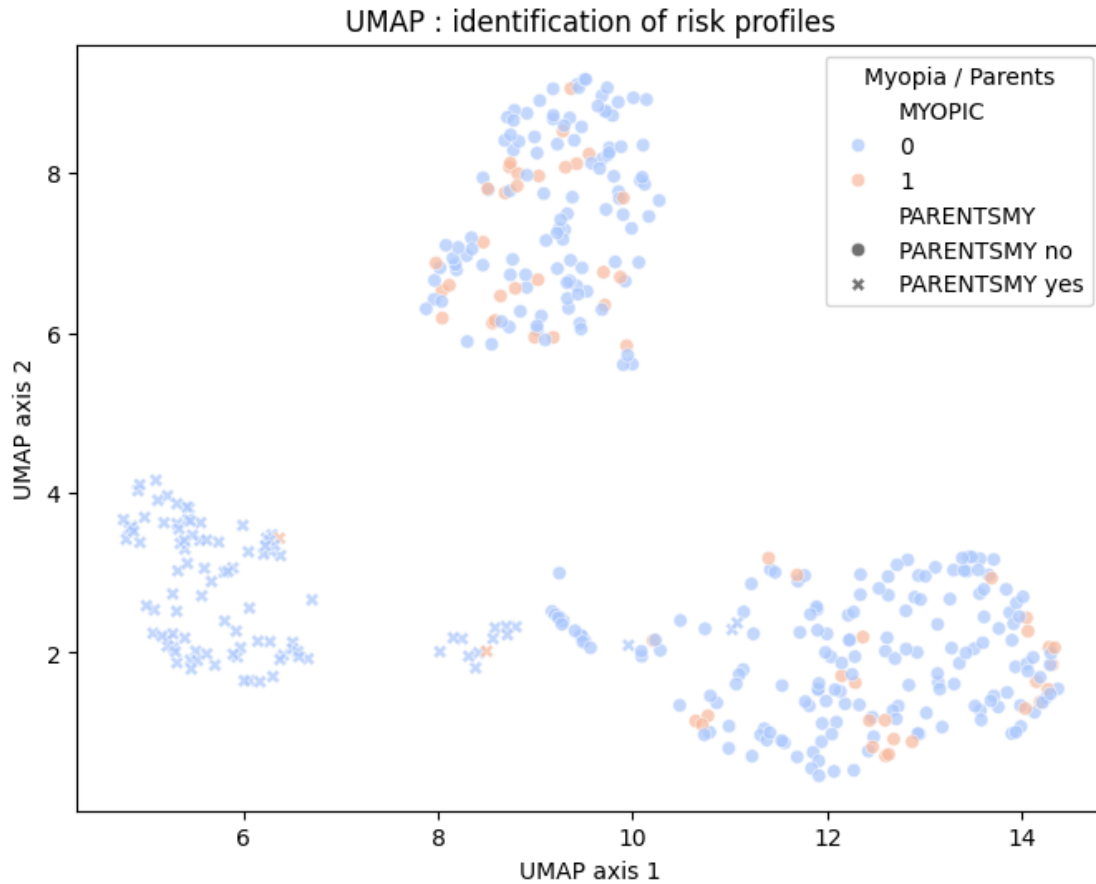
```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-
packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was
renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
```

```
warnings.warn(
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-
```

```
packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
```

```
warn(
```

```
OMP: Info #276: omp_set_nested routine deprecated, please use
omp_set_max_active_levels instead.
```



UMAP Projection: Visual Exploration of Risk Profiles The UMAP projection offers a 2D representation of the complex multi-feature space, aiming to highlight potential risk clusters among children.

In the scatterplot, each point represents an individual, with:

- **Color** indicating **myopic status** (MYOPIC: 0 = non-myopic, 1 = myopic)
- **Marker style** indicating **parental myopia** (PARENTSMY: yes/no)

While there is **no strict linear boundary** separating myopic from non-myopic cases, some **notable spatial patterns** can still be observed:

- In the **bottom left**, there appears to be a **clearer separation**, where most individuals have **parental myopia** (crosses) and are grouped together, suggesting a potential **familial or genetic influence**.

- In contrast, the **top right** and **bottom right** regions do not show any specific clustering pattern in terms of myopia status or parental history.
- Overall, myopic individuals (orange) remain **relatively dispersed** across the UMAP space, indicating **no single dominant linear boundary**, but potentially reflecting complex, multi-dimensional risk distributions.

However, since UMAP is **unsupervised and purely geometric**, it does not define explicit clusters. To further interpret these spatial structures, we will now apply **density-based clustering algorithms** such as OPTICS and DBSCAN.

From UMAP to Clustering: OPTICS & Reachability Plot To formalize the observation of groupings and **automatically detect potential risk clusters**, we apply the **OPTICS (Ordering Points To Identify the Clustering Structure)** algorithm on the UMAP-transformed space.

OPTICS is well-suited for this task because:

- It can **identify clusters of varying shape and density**
- It does not require a fixed number of clusters
- It naturally **detects noise/outliers**, which may correspond to **ambiguous or borderline cases**

We then analyze the **Reachability Plot**, a visual tool showing the “density distance” between points as OPTICS orders them. This plot helps to:

- Reveal **natural cluster boundaries**
- Suggest **possible values for eps** (similar to DBSCAN, but more flexible)
- Distinguish **dense clusters** (valleys) from **sparser/noisy regions** (peaks or plateaus)

This approach will allow us to determine if meaningful subgroups of children — potentially linked to **distinct risk profiles** — can be found **without supervision**, based purely on feature similarity.

```
[18]: cycol = cycle('bgrcmk')

# (1) computing reachability distance

clust = OPTICS(min_samples = 5).fit(X_train_umap)

plt.figure(figsize=(15, 10))
G = gridspec.GridSpec(2, 3)
ax1 = plt.subplot(G[0, :])

reachability = clust.reachability_[clust.ordering_]
# distance de 'reachabilité' -> à quel point il est accessible à partir d'un
↳ cluste dense
# + c'est bas, + le point est dans le cluster (sinon il devient isolé
```

```

labels = clust.labels_[clust.ordering_]
# étiquettes de cluster / labels

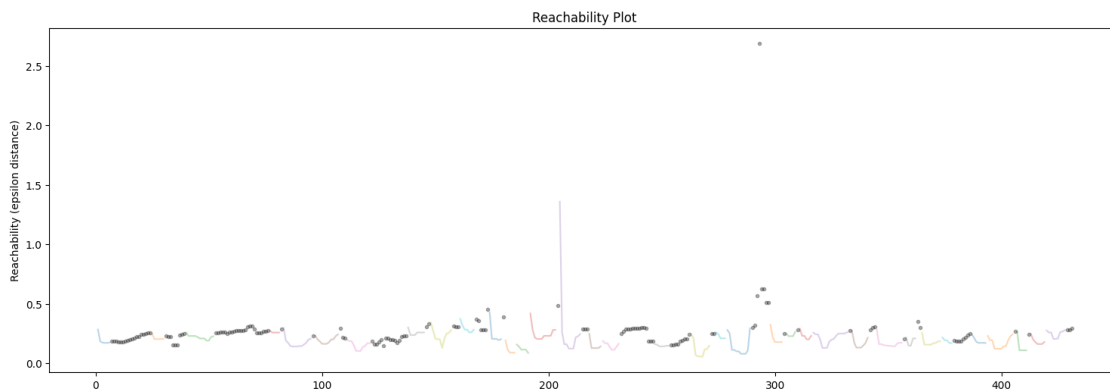
space = np.arange(len(X_train_umap))
# Abscisse -- chaque point aura une position unique sur l'axe horizontal (x)

# (2) Reachability plot
for i, klass in enumerate(set(clust.labels_)):
    if (klass == -1): continue
    Xk = space[labels == klass]
    Rk = reachability[labels == klass]
    ax1.plot(Xk, Rk ,alpha=0.3)

ax1.plot(space[labels == -1], reachability[labels == -1], 'k.', alpha=0.3)
#ax1.plot(space, np.full_like(space, eps_1, dtype=float), 'k-', alpha=0.5)
#ax1.plot(space, np.full_like(space, eps_2, dtype=float), 'k-', alpha=0.5)
ax1.set_ylabel('Reachability (epsilon distance)')
ax1.set_title('Reachability Plot')

plt.tight_layout()
plt.show()

```



Reachability Plot Analysis (OPTICS Clustering) The reachability plot provides a one-dimensional view of the **density-based structure** of the UMAP-transformed space, as captured by the OPTICS algorithm. Each valley (low reachability distance) corresponds to a **dense cluster**, while peaks and scattered points indicate **sparser regions or potential outliers**.

In this plot, we observe:

- Several **well-defined valleys** with low reachability distances, suggesting the presence of **multiple compact clusters**.
- The **height and width** of valleys vary, indicating that clusters differ in **density and size**, which justifies the use of OPTICS over DBSCAN.

- Points with **higher reachability values** (isolated dots above ~ 0.75) are likely **outliers or transitional cases**—potentially individuals who do not strongly belong to any risk group.

The absence of a flat or monotonic reachability pattern confirms that the dataset is **not uniformly dense** and contains **intricate substructures**, which could reflect **different clinical or genetic risk profiles**.

This visualization supports the idea that children can be grouped into **distinct density-based profiles**, some of which may correlate with myopic status or parental history.

Optimizing Cluster Extraction via Epsilon Sweeping (OPTICS + DBSCAN) Rather than directly mapping the OPTICS clustering results onto the UMAP projection, we perform a **systematic exploration of the epsilon parameter** using `cluster_optics_dbscan`, which extracts DBSCAN-like flat clusters from the OPTICS structure.

This process allows us to:

- Identify **stable and meaningful groupings**
- Control for **minimum cluster size**, and
- Evaluate clusters based on multiple criteria:
 - **Silhouette score** (cohesion vs separation)
 - **Entropy-based balance** (uniformity of cluster sizes)
 - **Custom clinical scores** (myopia-specific stratification quality)

For each value of `epsilon` in a finely spaced range, we:

- Extract clusters
- Filter noise and undersized clusters
- Compute metrics and combine them into a **composite score**

This enables selection of the **most promising clustering solutions**, balancing statistical robustness and clinical interpretability. Top candidate configurations are then **analyzed in detail** to identify cluster-specific myopia patterns — for instance, which clusters show higher or lower prevalence of myopia, and whether they align with known risk factors (e.g., parental myopia).

```
[19]: #min_cluster_size = 2
      #ideal_range = (1, 10) # nombre idéal de clusters pour toi ?
      group2 = df_train['MYOPIC'].map({0: 'MYOPIC yes', 1: 'MYOPIC no'})
      def optic_dbscan(X_train_umap, df_train, clust, eps_values=np.linspace(0, 2, 200), min_cluster_size=2):

          def normalized_entropy(counts):
              sizes = np.array(list(counts.values()))
              if len(sizes) <= 1:
                  return 0.0
              p = sizes / sizes.sum()
              return entropy(p, base=len(p)) # entre 0 (déséquilibré) et 1 (équilibré)
```

```

results = []
scores = []
silhouettes = []
balances = []
n_clusters_list = []
valid_eps = []
list_labels = []

for i, eps in enumerate(eps_values):
    # Clusterisation via DBSCAN-like interpretation d'OPTICS
    labels = cluster_optics_dbscan(
        reachability=clust.reachability_,
        core_distances=clust.core_distances_,
        ordering=clust.ordering_,
        eps=eps
    )
    labels = np.array(labels)

    # Comptage de tous les clusters, y compris bruit
    clusters = Counter(labels)
    n_clusters = len([k for k in clusters if k != -1])

    # Calcul de l'équilibre
    balance = 1 - normalized_entropy(clusters)

    # On retire les points de bruit pour calculer silhouette
    mask = labels != -1
    filtered_X = X_train_umap[mask]
    filtered_labels = labels[mask]

    try:
        silhouette = silhouette_score(filtered_X, filtered_labels)
        final_score = silhouette * (0.5 + 0.5 * balance) # pondération
    except:
        # simple

    # Analyse clinique des clusters
    myopic_cluster = []
    for cluster in sorted(set(filtered_labels)):
        if cluster == -1:
            continue
        cluster_mask = (labels == cluster)
        total = cluster_mask.sum()
        myopic_yes = ((group2 == 'MYOPIC no') & cluster_mask).sum()
        pourcent = myopic_yes / total * 100 if total > 0 else 0
        myopic_cluster.append({
            'cluster': cluster,
            'n_total': total,

```

```

        'n_myopic_no': myopic_yes,
        '%_myopic_no': round(pourcent, 1)
    })

summary_df = pd.DataFrame(myopic_cluster).sort_values(by='cluster')

# Stockage des résultats
result = {
    "eps": eps,
    "indx": i,
    "score": final_score,
    "silhouette": silhouette,
    "balance": balance,
    "n_clusters": n_clusters,
    "clusters": summary_df
}

results.append(result)
scores.append(final_score)
silhouettes.append(silhouette)
balances.append(balance)
n_clusters_list.append(n_clusters)
valid_eps.append(eps)
list_labels.append(labels)

except Exception as e:
    continue

return results, scores, silhouettes, n_clusters_list, valid_eps, balances, \
↳ list_labels

results, scores, silhouettes, n_clusters_list, valid_eps, balances, list_labels, \
↳ = optic_dbscan (X_train_umap, df_train, clust)
plt.figure(figsize=(14,5))

plt.subplot(1,3,1)
plt.plot(valid_eps, silhouettes, marker='o', label='Silhouette')
plt.plot(valid_eps, scores, marker='x', label='Final Score')
plt.xlabel("Epsilon")
plt.title("Silhouette & Final Score")
plt.legend()

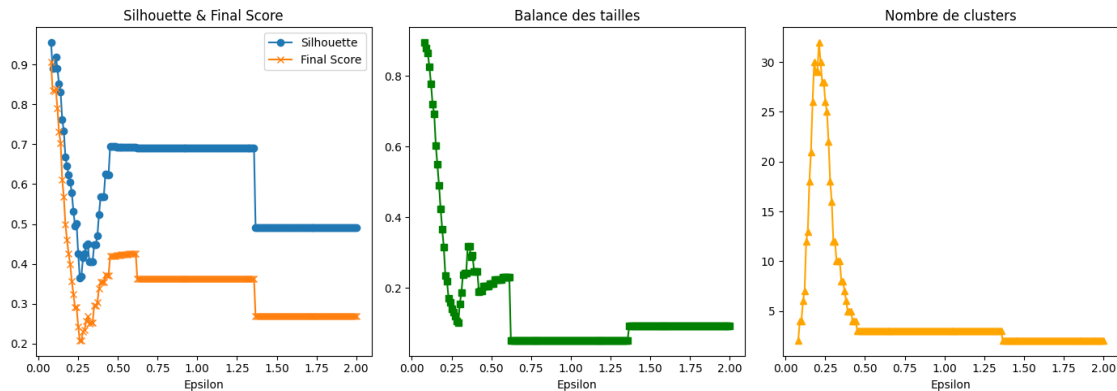
plt.subplot(1,3,2)
plt.plot(valid_eps, balances, marker='s', label='Balance (1 - entropy)', \
↳ color='green')
plt.xlabel("Epsilon")

```

```
plt.title("Balance des tailles")

plt.subplot(1,3,3)
plt.plot(valid_eps, n_clusters_list, marker='^', label='Nb clusters',
        color='orange')
plt.xlabel("Epsilon")
plt.title("Nombre de clusters")

plt.tight_layout()
plt.show()
```



```
[20]: def compute_scores(n_clusters_list, summary_eps, silhouettes, balances,
        valid_eps):
    # Préparation pour normalisation
    max_sil = max([silhouettes[i] for i in range(len(summary_eps))])
    max_bal = max([balances[i] for i in range(len(summary_eps))])

    max_myopic = max([get_myopic_score(df) for df in summary_eps])

    results_ = []

    for indx, df_ in enumerate(summary_eps):
        # Skip si bruit trop élevé
        #if df_['n_myopic_no'][df_['cluster'] == -1].iloc[0] >= 80:
        #    continue

        sil = silhouettes[indx]
        bal = balances[indx]
        myopic = get_myopic_score(df_)

        norm_sil = sil / max_sil if max_sil else 0
        norm_bal = bal / max_bal if max_bal else 0
```

```

norm_myo = myopic / max_myopic if max_myopic else 0

final_score = (norm_sil + norm_bal + norm_myo) / 3

results_.append({
    "index": indx,
    "eps": valid_eps[indx],
    "silhouette": sil,
    "balance": bal,
    "myopic_score": myopic,
    "score": final_score,
    "n_clusters": n_clusters_list[indx]
})

return pd.DataFrame(results_).sort_values(by="score", ascending=False)

```

```

[22]: summary_eps = []
for i in range(len(results)): summary_eps.append(results[i]['clusters'])
compute_scores(n_clusters_list, summary_eps, silhouettes, balances, valid_eps).
    ↪head()

```

```

[22]:
   index  eps  silhouette  balance  myopic_score  score  n_clusters
3      3  0.110553   0.919599  0.824188    19.766667  0.790622         6
5      5  0.130653   0.851543  0.719385    26.525000  0.783607        12
2      2  0.100503   0.893129  0.864665    16.700000  0.771178         4
6      6  0.140704   0.831584  0.692349    25.275000  0.756267        13
4      4  0.120603   0.889984  0.776807    18.700000  0.753850         7

```

```

[26]: display(results[3]['clusters']), results[3]['eps']

```

	cluster	n_total	n_myopic_no	%_myopic_no
0	0	4	1	25.0
1	1	4	0	0.0
2	2	2	0	0.0
3	3	5	0	0.0
4	4	7	1	14.3
5	5	5	1	20.0

```

[26]: (None, 0.11055276381909548)

```

```

[24]: eps_1 = results[3]['eps']
print("eps1 :", eps_1)

labels_eps_1 = cluster_optics_dbscan(reachability=clust.reachability_,
                                     core_distances=clust.core_distances_,
                                     ordering=clust.ordering_, eps=eps_1)

```

```

from matplotlib.lines import Line2D
plt.figure(figsize=(6, 12))

custom_legend = [
    Line2D([0], [0], marker='s', color='gray', label='MYOPIC yes',
           markerfacecolor='gray', markersize=8, linestyle='None'),
    Line2D([0], [0], marker='^', color='gray', label='MYOPIC no',
           markerfacecolor='gray', markersize=8, linestyle='None')
]

plt.legend(handles=custom_legend, loc='lower center', ncol=2, bbox_to_anchor=(0.
    ↪5, -0.05))

G = gridspec.GridSpec(2, 1) # 3 lignes, 1 colonne
ax2 = plt.subplot(G[0, 0])
ax3 = plt.subplot(G[1, 0])

for i, klass in enumerate(set(clust.labels_)):
    if (klass == -1): continue
    Xk = X_train_umap[clust.labels_ == klass]
    ax2.scatter(Xk[:, 0], Xk[:, 1])
ax2.plot(X_train_umap[clust.labels_ == -1, 0], X_train_umap[clust.labels_ == -1, 1], 'k+', alpha=0.1)
ax2.set_title('Automatic Clustering\nOPTICS')

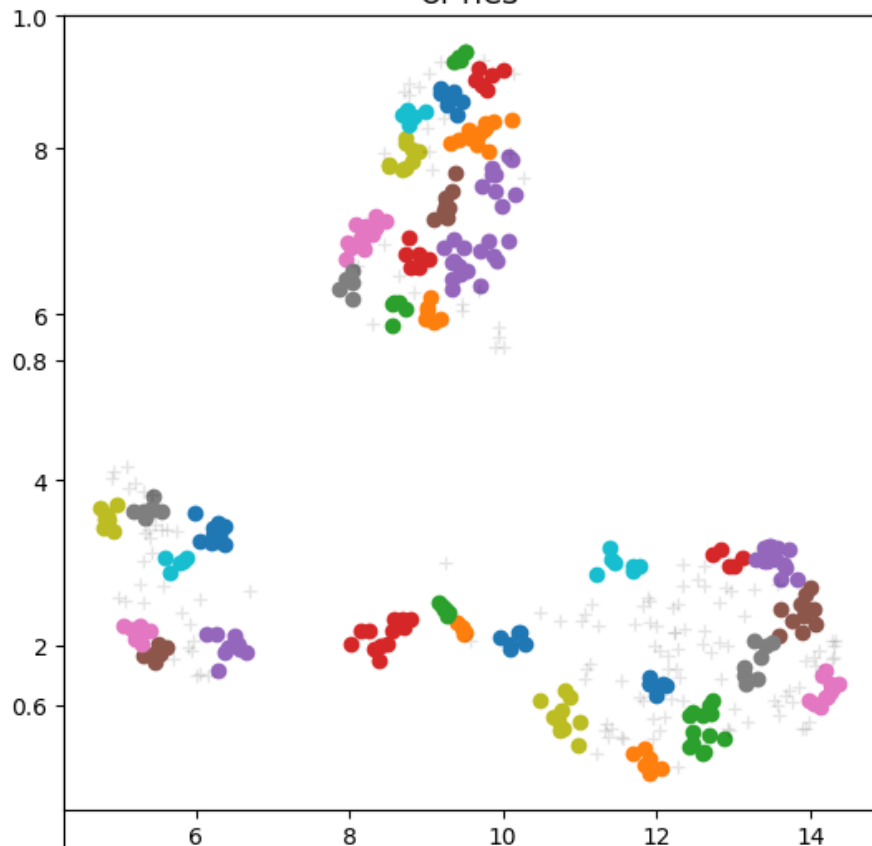
# (4) DBSCAN, epsilon = eps_1
plot_optics_dbscan_clusters(X_train_umap, labels_eps_1, group2, ax3,
    ↪f'Clustering at {eps_1} epsilon cut\nDBSCAN')

plt.tight_layout()
plt.show()

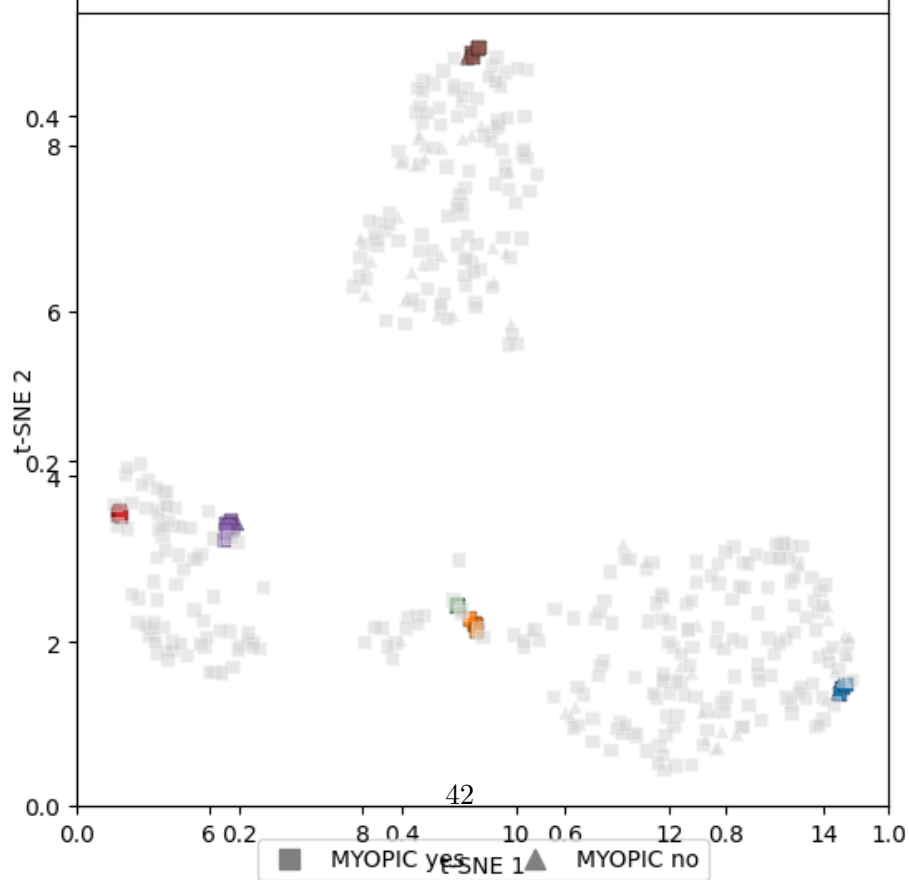
```

eps1 : 0.11055276381909548

Automatic Clustering
OPTICS



Clustering at 0.11055276381909548 epsilon cut
DBSCAN



OPTICS with UMAP: Clustering Interpretation

- **Cluster composition and clinical insight** During the *epsilon* optimization process, we were unable to identify any clustering solution that provided both a **high silhouette score** and a **good global clustering quality** with few outliers. Most configurations that yielded acceptable cluster definitions resulted in a **large number of outliers**, indicating that the data does not naturally separate into distinct clusters in the UMAP-reduced space.

This suggests that, within this dataset, **meaningful clustering is not achievable**, at least with the combination of **UMAP dimensionality reduction** and **OPTICS density-based clustering**. The underlying data structure may be too **complex, overlapping, or continuous** to form well-defined groups.

4.1 2.1. Statistic analysis

4.1.1 2.1.1. Univariate Analysis

- Distribution of continuous features (boxplot and histogram).
- Categorical features breakdown.

```
[27]: cat = ['MYOPIC', 'GENDER', 'PARENTSMY']
myopianum = df_train.drop(cat, axis=1)
myopiafact = df_train[cat]
```

```
[28]: myopianum.head(5)
```

```
[28]:
```

	AGE	SPHEQ	AL	ACD	LT	VCD	SPORTHR	SCREENHR	CLOSEHR
0	6	0.326	23.29	4.013	3.478	15.80	12	8	17
1	6	1.272	22.12	4.048	3.542	14.53	6	10	34
2	6	1.044	22.00	3.516	3.612	14.88	12	12	42
3	6	-0.207	21.82	3.904	3.450	14.47	20	4	32
4	6	-0.175	23.67	3.865	3.245	16.56	9	19	24

```
[29]: myopiafact.head(5)
```

```
[29]:
```

	MYOPIC	GENDER	PARENTSMY
0	0	0	1
1	0	0	1
2	0	0	1
3	1	1	1
4	0	0	1

4.1.2 Univariate Analysis and Multivariate Visualization

- **Distribution of Continuous Features:** Visualized using boxplots and histograms.
- **Categorical Features Breakdown:** Analyzed to understand their distribution and impact.

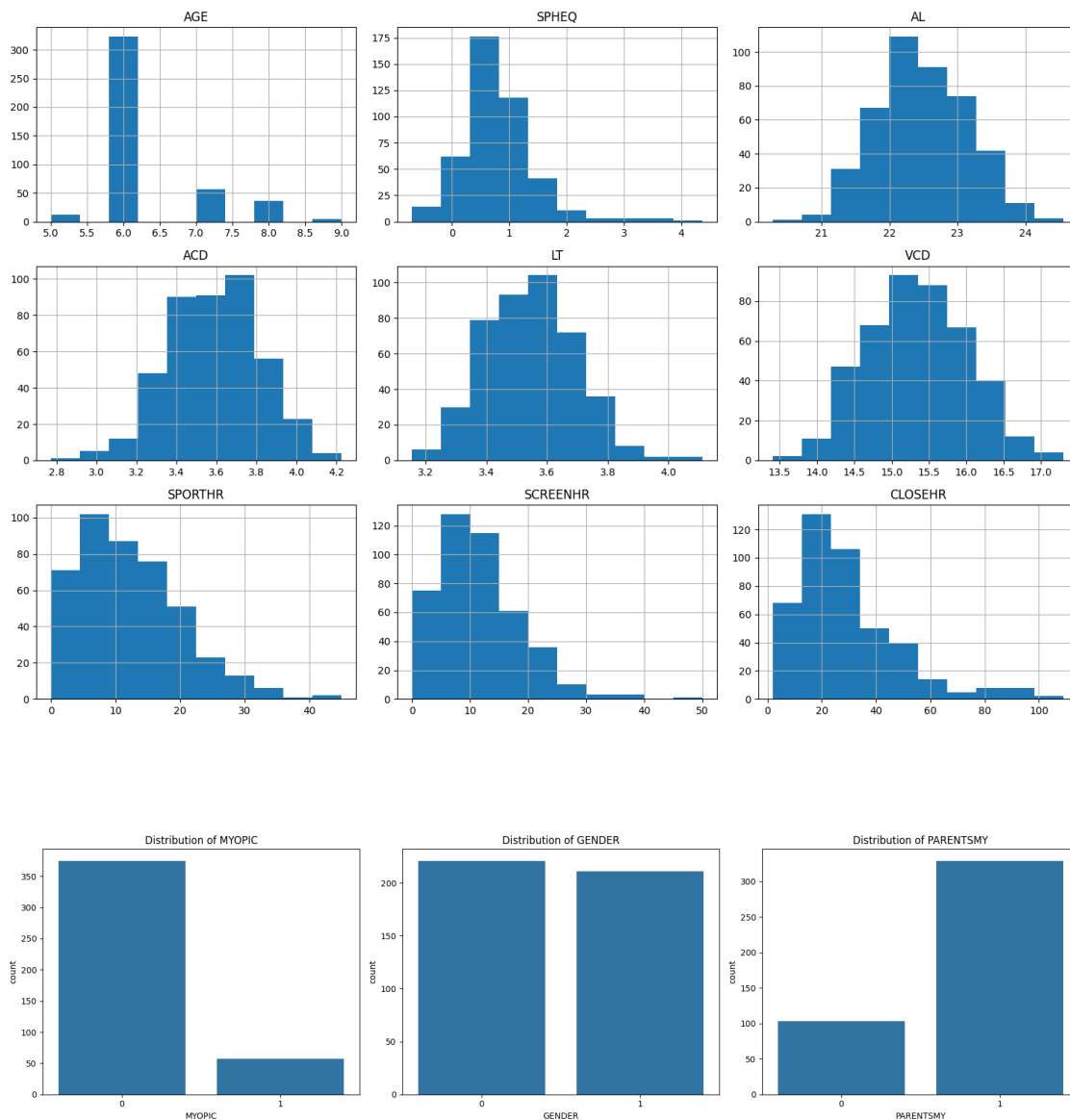
```
[35]: myopianum.hist(figsize=(15,10))
plt.tight_layout()
plt.show()

categorical_cols = ['MYOPIC', 'GENDER', 'PARENTSMY']

fig, axes = plt.subplots(1, 3, figsize=(18,5))

for i, col in enumerate(categorical_cols):
    sns.countplot(x=col, data=myopiafact, ax=axes[i])
    axes[i].set_title(f'Distribution of {col}')

plt.tight_layout()
plt.show()
```



```
[31]: num_cols = df_train.drop(cat, axis=1).columns.tolist()
      for col in num_cols:
          df_train[col] = pd.to_numeric(df_train[col], errors='coerce')

      df_train['GENDER'] = df_train['GENDER'].astype('category')
      df_train['PARENTSMY'] = df_train['PARENTSMY'].astype('category')
      df_train['MYOPIC'] = df_train['MYOPIC'].astype('category')

[42]: fig = go.Figure(data=[
      go.Bar(name='GENDER',
              x=['Males', 'Females'],
              y=table_genre.loc['Myopic'].values*100),
      go.Bar(name='PARENTMY',
              x=['No Myopic Parents', 'At least one Myopic Parent'],
              y=table_Parents.loc['Myopic'].values*100)
    ])
fig.update_layout(barmode='group', yaxis_title="Percentage of Myopic(%)")
fig.show()
```

Chi-Square Test (Chi²) The **Chi-Square test** evaluates whether there is a significant association between two categorical variables. It compares the **observed frequencies** in each category to the **expected frequencies** under the assumption of independence. • **Calculation:** $\sum ((\text{observed} - \text{expected})^2 / \text{expected})$ across all cells of the contingency table. • **Interpretation:** A high Chi² value (with $p < 0.05$) suggests that the variables are not independent (i.e., they are associated).

Fisher's Exact Test The **Fisher's Exact Test** is used to determine if there are non-random associations between two categorical variables, especially when **sample sizes are small** or when expected frequencies are < 5 . • **Calculation:** Uses hypergeometric probability to calculate the exact probability of obtaining the observed distribution, under the null hypothesis. • **Interpretation:** A p-value < 0.05 indicates a significant association. Fisher is preferred over Chi² for small sample sizes due to its exact nature.

Odds Ratio (OR) The **Odds Ratio** quantifies the strength of association between an exposure and an outcome. It is the ratio of the odds of the event occurring in the exposed group to the odds in the non-exposed group. • **Calculation:** $OR = (a/b) / (c/d) = (ad)/(bc)$, where a, b, c, d are cell counts in a 2x2 table. • **Interpretation:**

- $OR = 1 \rightarrow$ no association
- $OR > 1 \rightarrow$ exposure increases odds of outcome
- $OR < 1 \rightarrow$ exposure decreases odds of outcome

Cramer's V **Cramer's V** measures the **strength of association** between two categorical variables, adjusting for table size. It ranges from 0 (no association) to 1 (perfect association). • **Calculation:** $V = \sqrt{(\chi^2 / [n * (k - 1)])}$, where χ^2 is the chi-square statistic, n is total sample size, and k is the smaller number of categories (rows or columns).

• **Interpretation:**

- $V < 0.1 \rightarrow$ negligible association
- $0.1 \leq V < 0.3 \rightarrow$ weak association
- $0.3 \leq V < 0.5 \rightarrow$ moderate association
- $V \geq 0.5 \rightarrow$ strong association

```
[41]: # ----- GENDER -----
print("MYOPIC x GENDER")
print('Nb Boys : ', (df_train['GENDER']==0).sum())
print('Nb Girls : ', (df_train['GENDER']==1).sum())

# Crosstab
table1 = pd.crosstab(df_train['MYOPIC'], df_train['GENDER'])
display(table1)

# Chi-2 test
chi2, p, dof, ex = stats.chi2_contingency(table1)
print('Chi-2 p-value:', p)
print('Chi-2 value:', chi2)
print('Chi-2 dof:', dof)

# Fisher's Exact Test
try:
    oddsratio, p_fisher = fisher_exact(table1)
    print("Fisher's Exact Test p-value:", p_fisher)
    print("Odds Ratio:", oddsratio)
except:
    print("Fisher's Exact Test not applicable for this table shape.")

# Cramer's V
n = table1.sum().sum()
cramers_v = np.sqrt(chi2 / (n * (min(table1.shape)-1)))
print("Cramer's V:", crammers_v)

# Interprétation Cramer's V
if crammers_v < 0.1:
    interpretation = "Negligible association"
elif crammers_v < 0.3:
    interpretation = "Weak association"
elif crammers_v < 0.5:
    interpretation = "Moderate association"
else:
    interpretation = "Strong association"
print("Interpretation (Cramer's V):", interpretation)

# Normalized crosstab
table_genre = pd.crosstab(df_train['MYOPIC'], df_train['GENDER'],
                           normalize='columns').round(2)
```

```

table_genre.index = ['No-Myopic', 'Myopic']
table_genre.columns = ['Man', 'Woman']
display(table_genre)

# ----- PARENTMY -----
print("\n", "-" * 30, "\nMYOPIC x PARENTSMY")
print('Nb Myopic Parents : ', (df_train['PARENTSMY']==1).sum())
print('Nb Non Myopic Parents : ', (df_train['PARENTSMY']==0).sum())

# Crosstab
table2 = pd.crosstab(df_train['MYOPIC'], df_train['PARENTSMY'])
display(table2)

# Chi-2 test
chi2, p, dof, ex = stats.chi2_contingency(table2)
print('Chi-2 p-value:', p)
print('Chi-2 value:', chi2)
print('Chi-2 dof:', dof)

# Fisher's Exact Test
try:
    oddsratio, p_fisher = fisher_exact(table2)
    print("Fisher's Exact Test p-value:", p_fisher)
    print("Odds Ratio:", oddsratio)
except:
    print("Fisher's Exact Test not applicable for this table shape.")

# Cramer's V
n = table2.sum().sum()
cramers_v = np.sqrt(chi2 / (n * (min(table2.shape)-1)))
print("Cramer's V:", crammers_v)

# Interprétation Cramer's V
if crammers_v < 0.1:
    interpretation = "Negligible association"
elif crammers_v < 0.3:
    interpretation = "Weak association"
elif crammers_v < 0.5:
    interpretation = "Moderate association"
else:
    interpretation = "Strong association"
print("Interpretation (Cramer's V):", interpretation)

# Normalized crosstab
table_Parents = pd.crosstab(df_train['MYOPIC'], df_train['PARENTSMY'],
                             normalize='index').round(2)

```

```
table_Parents.index = ['No-Myopic', 'Myopic']
table_Parents.columns = ['No', 'Yes']
display(table_Parents)
```

MYOPIC x GENDER

Nb Boys : 221

Nb Girls : 211

GENDER	0	1
--------	---	---

MYOPIC

0	196	179
---	-----	-----

1	25	32
---	----	----

Chi-2 p-value: 0.29795054292322165

Chi-2 value: 1.0833446284321815

Chi-2 dof: 1

Fisher's Exact Test p-value: 0.25729726261176894

Odds Ratio: 1.401564245810056

Cramer's V: 0.05007736210549498

Interpretation (Cramer's V): Negligible association

	Man	Woman
No-Myopic	0.89	0.85
Myopic	0.11	0.15

MYOPIC x PARENTSMY

Nb Myopic Parents : 329

Nb Non Myopic Parents : 103

PARENTSMY	0	1
-----------	---	---

MYOPIC

0	101	274
---	-----	-----

1	2	55
---	---	----

Chi-2 p-value: 0.00021562337360903677

Chi-2 value: 13.689791775451848

Chi-2 dof: 1

Fisher's Exact Test p-value: 1.8652094527211244e-05

Odds Ratio: 10.136861313868613

Cramer's V: 0.17801497918306186

Interpretation (Cramer's V): Weak association

	No	Yes
No-Myopic	0.27	0.73
Myopic	0.04	0.96

crosstab	chi2 p-value	chi2	Fisher p-value	Odd Ratio	Cramer's V	Proportion Analysis
Myopic- Gender	0.3	1.1	0.25	1.4	0.05	the both p-value above 0.05. Odd Ratio is a bit more present for females (but because of small dataset), and cramer's V small. It implies that there is no statistically significant association between gender and myopia.

crosstab	chi2 p-value	chi2	Fisher p-value	Odd Ratio	Cramer's V	Proportion Analysis
Myopic- Parentsmy	2.1 E-5	13.2	1.9E-5	10.14	0.17	both p-value are very small (E-5), chi2 and Odd Ratio very high (Myopic Parent's children have 10 more chance tp be myopic (Odd Ratio), and coherence for Odd Ratio and Cramer's V. Children with at least one myopic parent are much more likely to be myopic themselves.

4.1.3 2.1.2. Quant values

T-Test analysis The t-test evaluates whether two groups have significantly different means. A **negative t-statistic** indicates that the mean of the first group is lower than the second. The **p-value** quantifies the likelihood that this difference is due to chance. A **small p-value (typically < 0.05)** suggests a statistically significant difference between groups, regardless of the sign of the t-statistic.

```
[43]: for col in myopianum.columns:
        group0 = df_train.loc[df_train['MYOPIC'] == 0, col]
        group1 = df_train.loc[df_train['MYOPIC'] == 1, col]
        stat, p = stats.ttest_ind(group0, group1, nan_policy='omit')
```

```
fig = px.box(df_train, x='MYOPIC', y=col, color='MYOPIC', points="all",
title=col)
fig.show()
print(f"T-test {col}: statistic={stat:.2f}, p-value={p:.4f} \n-----")
```

T-test AGE: statistic=0.41, p-value=0.6815

T-test SPHEQ: statistic=8.91, p-value=0.0000

T-test AL: statistic=-0.46, p-value=0.6479

T-test ACD: statistic=-2.67, p-value=0.0078

T-test LT: statistic=1.50, p-value=0.1345

T-test VCD: statistic=0.11, p-value=0.9125

T-test SPORTHR: statistic=2.13, p-value=0.0335

T-test SCREENHR: statistic=0.40, p-value=0.6871

T-test CLOSEHR: statistic=0.19, p-value=0.8506

Col	NonMyopic BoxPlot	Myopic Boxplot	Test-T – p-value	Test-T – statistic	Conclusion
SPORTHR	q1 6, median 10, q3 16	q1 3, median 8, q3 15	p-value = 0.0335	2.13	Myopia ⇒ less sport
ACD	q1 3.42, median 3.57, q3 3.53	q1 3.52, median 3.69, q3 3.8	p-value = 0.0078	-2.67	Myopia ⇒ less sport
SPHEQ	q1 0.545, median 0.791, q3 1.097	q1 -0.0735, median 0.234, q3 0.507	p-value = 0.0000	8.91	Myopia ⇒ lower SPHEQ

Statistical Comparison of Myopic and Non-Myopic Groups

```
[44]: corr = myopianum.corr()
plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()

# Suppression des doublons et diagonales
```

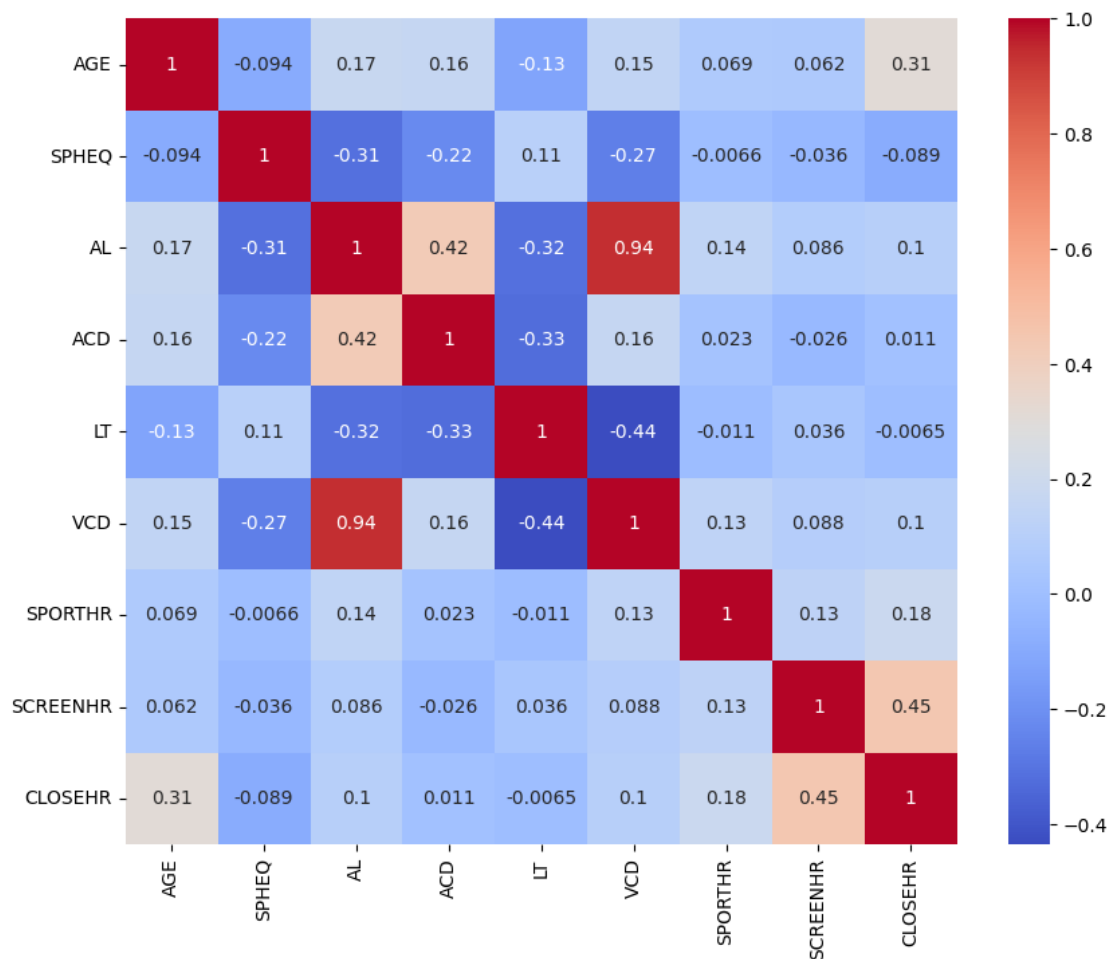
```

corr_pairs = (
    corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
        .stack()
        .reset_index()
)
corr_pairs.columns = ['Variable 1', 'Variable 2', 'Correlation']

# Tri du plus faible au plus fort
corr_pairs_sorted = corr_pairs.reindex(corr_pairs['Correlation'].abs()
    ↪sort_values(ascending=False).index)

high_correlation = corr_pairs_sorted[corr_pairs_sorted['Correlation'].abs() > 0.
    ↪4 ]
display(high_correlation)
display(Markdown("***Sorted correlations between variables (excluding_
    ↪self-correlations)***"))

```



	Variable 1	Variable 2	Correlation
17	AL	VCD	0.942082
35	SCREENHR	CLOSEHR	0.449825
26	LT	VCD	-0.435219
15	AL	ACD	0.423216

Sorted correlations between variables (excluding self-correlations)

```
[45]: # Création d'une liste unique des variables fortement corrélées
var = high_correlation['Variable 1'].unique().tolist() + [
    val for val in high_correlation['Variable 2'].unique()
    if val not in high_correlation['Variable 1'].unique()
]
sns.pairplot(df_train, hue='MYOPIC', vars=var)
plt.suptitle("Pairplot key variables", y=1.02)
plt.show()
```



4.2 Conclusion and Synthesis

- **Independancy**
 - The variable **PARENTSMY** and **Myopic** are not completely independant.
- **Key Discriminant Variable:**
 - The variable **SPHEQ** (spherical equivalent) displays the strongest difference between myopic and non-myopic children, both statistically and visually.
 - **ACD** (anterior chamber depth) also shows a significant difference, reinforcing the role of ocular morphology in myopia risk.
 - **SPORTHR** (hours of sport) is lower among myopic individuals.
- **Correlations:**
 - The highest correlation found is between **AL** and **VCD**, indicating a shared underlying structural influence.
 - Other strong correlations include **SCREENHR-CLOSEHR**, **LT-ACD** and **LT-VCD**, helping explain possible latent patterns in biometric or behavioral variables.

5 4. Global Synthesis and Recommendations

5.1 Key Analytical Findings

- **Family History is a Significant but Non-Deterministic Factor:** While **parental myopia (PARENTSMY)** is significantly associated with myopia risk, it is not an absolute determinant. Many children with myopic parents do not develop myopia, and some children without myopic parents do, underscoring the multifactorial nature of the condition.
- **SPHEQ Dominates as a Discriminant Variable:** The **spherical equivalent (SPHEQ)** is the strongest differentiator between myopic and non-myopic children, both statistically and clinically. **ACD** (anterior chamber depth) also demonstrates a notable difference, highlighting the importance of ocular morphology in myopia risk.
- **Physical Activity Shows Limited but Consistent Association:** Myopic children tend to report slightly fewer hours of sport (**SPORTHR**), though the practical significance of this difference remains modest.
- **Key Correlations Suggest Latent Structures:** Strong correlations were identified between **AL-VCD**, **SCREENHR-CLOSEHR**, and **LT-ACD/VCD**, reflecting shared structural or behavioral influences that should be considered in multivariate models to avoid redundancy.
- **No Meaningful Gender or Screen Time Effects Detected:** Gender and self-reported screen or near-work hours did not show significant associations with myopia in this dataset.

5.2 UMAP + OPTICS Clustering Insights

- **Lack of Distinct Clusters:** Despite extensive *epsilon* optimization, no clustering solution achieved both a **high silhouette score** and **robust global structure** with minimal outliers. Solutions with clearer clusters produced a high proportion of outliers, indicating that the data lacks natural discrete groupings in the UMAP-reduced space.

This suggests the underlying data is **continuous, overlapping, or too complex** to yield meaningful clusters using **UMAP + OPTICS**. Alternative approaches or additional data dimensions may be required to uncover latent subgroups.

5.3 Strategic Recommendations

1. **Maximize Sensitivity to Detect Myopia Risk:** Prioritize class weighting, SMOTE, and threshold tuning to improve recall for the minority class while monitoring precision.
2. **Expand Feature Engineering Efforts:** Develop and test interaction and nonlinear features (e.g., SPHEQ $\times$ SPORTHR, PARENTSMY $\times$ SPHEQ) to capture multifactorial risk patterns.
3. **Continuously Monitor Model Fairness:** Evaluate performance across demographic and clinical subgroups to ensure equity and identify potential biases in predictions.